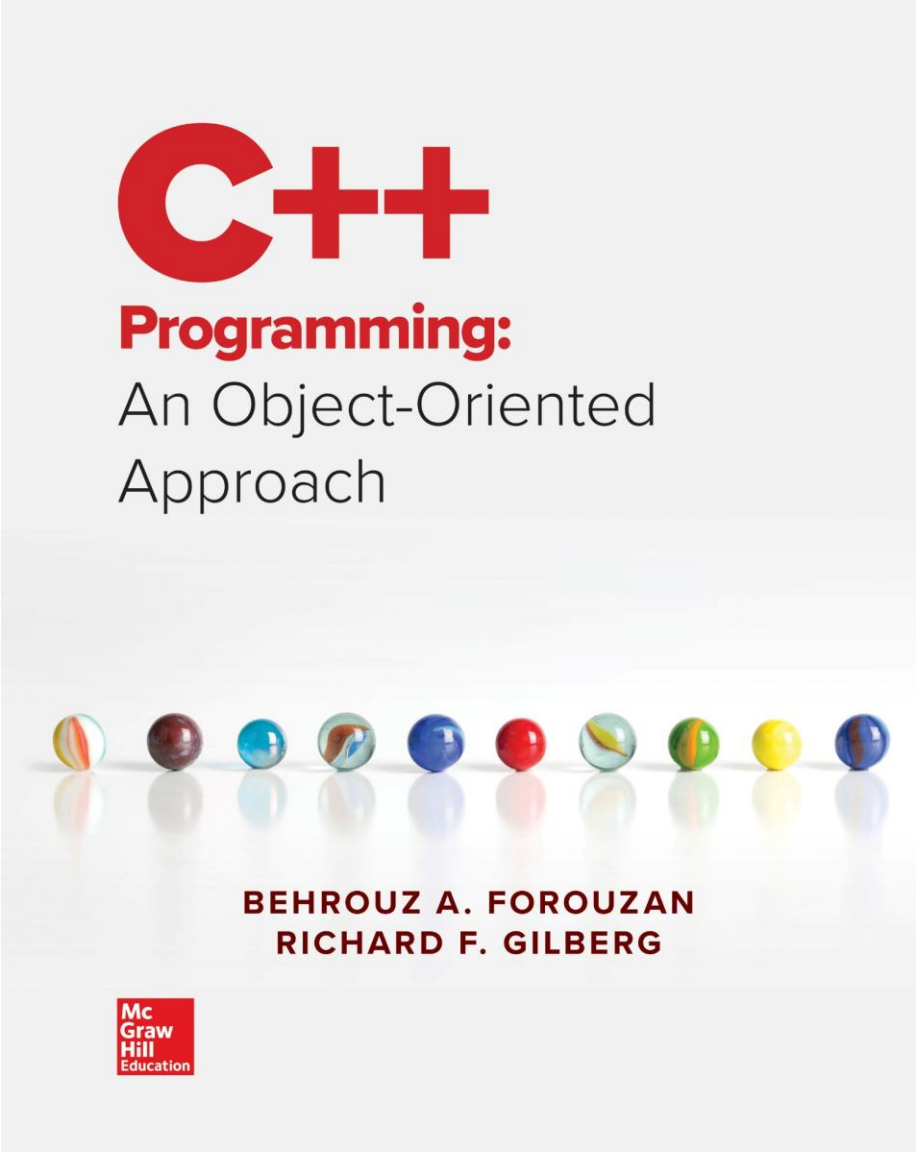
C++ Programming:

An Object-Oriented Approach

Chapter 9

References, Pointers, and Memory Management



BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

©2020 McGraw-Hill Education. All rights reserved. Authorized only for instructor use in the classroom. No reproduction or further distribution permitted without the prior written consent of McGraw-Hill Education.

Objectives Part 1

To discuss reference types and reference variables.

To discuss how to create permanent binding between a reference variable and the original variable.

To discuss how to retrieve and change data in the original variable through the reference variable.

To show the application of references in relation to functions: pass-by-reference and return-by-reference.

©2020 McGraw-Hill Education

Objectives Part 2

To discuss pointer types and pointer variables.

To discuss how to create permanent or temporary binding between a pointer variable and a data variable.

To discuss how to retrieve and change data in a data variable through the pointer variable.

To show the application of pointers in relation to functions: pass-by-pointer and return-by-pointer.

©2020 McGraw-Hill Education

Objectives Part 3

To discuss relations between arrays and pointers and to discuss array arithmetic.

To discuss four areas of memory available to programmers and their applications.

To discuss when an object can be stored in stack memory and when it needs to be stored in heap memory.

To discuss dynamic memory allocation.

©2020 McGraw-Hill Education

REFERENCES

A reference is an alternative name for an object.

References have been added to C++ to simplify communication between entities such as functions as we see later in the chapter.

©2020 McGraw-Hill Education

Introduction Part 1

A variable declared as *type*& is an alternative name for a variable declared as *type*.

When we declare a reference variable, we do not create a new object in memory; we just declare an alternative name for an existing variable.

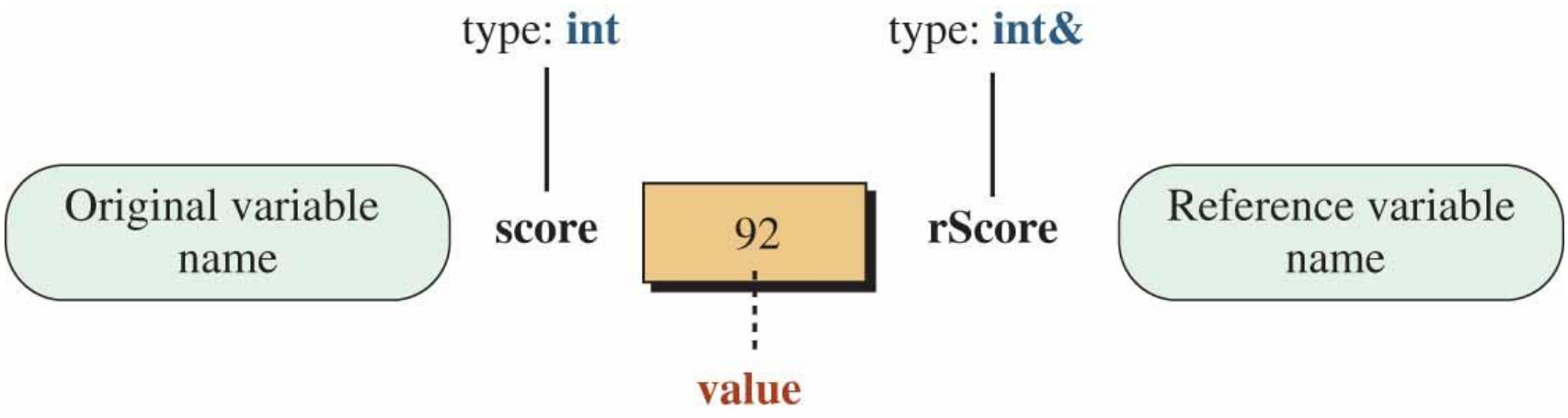
For this reason, we need to *bind* a reference variable to the original variable immediately when we declare it.

```
int score = 92;          // Declaring and initializing score
int&  rScore = score;    // Declaring rScore and binding it to score
```


©2020 McGraw-Hill Education

Introduction Part 2

Figure 9.1 *Original variable and reference variable*



©2020 McGraw-Hill Education

Introduction Part 3

Compound Type.

Although the original variable and the reference variable define the same location in memory with the same value seen by both of them, the types of the original variable and the reference variable are different.

For example the type of variable *score* is *int*, but the type of variable *rScore* is *int*&.

In other words, the use of reference variables creates a new compound type: *reference* type.

A reference type is a compound type because we can have a reference to an *int* (*int*&), a reference to a *double* (*double*&), a reference to a *bool* (*bool*&), and so on.

©2020 McGraw-Hill Education

Introduction Part 4

Compound Type.

When we bind a variable to a reference variable, the type of the reference variable should be a reference to the type of the original variable.

The following is a compilation error.

We cannot initialize a reference variable of *double*& to refer to a variable of type *int*.

```
int num = 100;
double& rNum = num; // Compilation error. Type mismatch
```


©2020 McGraw-Hill Education

Introduction Part 5

Permanent Binding.

After a reference variable has been declared and bound to a variable name, the reference relationship has been created between them and cannot be broken until the variables are destroyed (go out of scope).

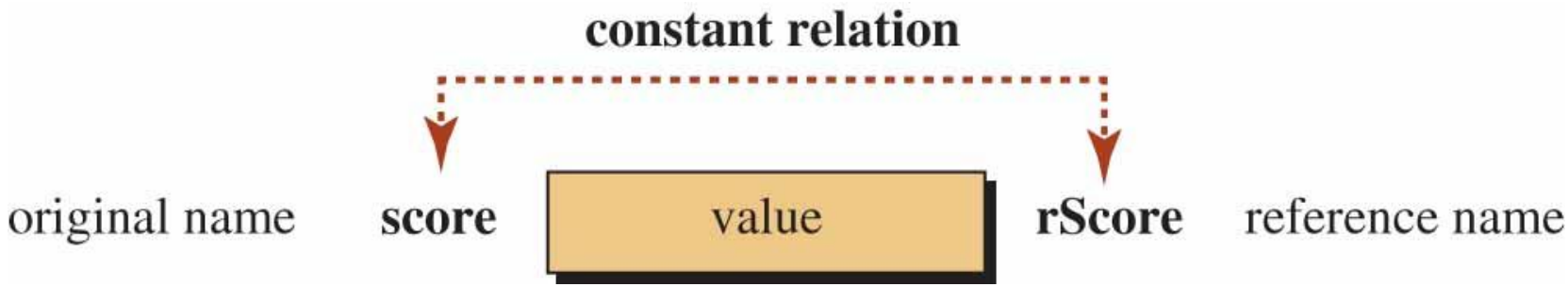
In C++ parlance, we say that the relationship between a variable and the corresponding reference variable is *constant*.

Figure 9.2 shows that we cannot change the reference relation after it has been created.

©2020 McGraw-Hill Education

Introduction Part 6

Figure 9.2 ***The reference relation as a constant relation***



After the reference relationship is established, it cannot be changed.

©2020 McGraw-Hill Education

Introduction Part 7

Example 9.1

We will get a compilation error if we first bind *rScore* to *score* and then try to break this relationship and bind *rScore* to *num* as shown below:

```
int score = 92;
int& rScore = score;
int num = 80;
int& rScore = num; // error, breaking the reference relationship
```


©2020 McGraw-Hill Education

Introduction Part 8

Example 9.2

Sometimes we see statements that should not be confused with breaking the reference relationship as shown below:

```
int score = 92;
int& rScore = score;
int num = 80;
rScore = num;
```

In this code segment, we are not breaking the relationship.

We are storing a copy of value in *num* to the common memory location established by the reference relationship.

In other words, the last statement is not binding; it is an assignment to the common variable.

©2020 McGraw-Hill Education

Introduction Part 9

Multiplicity

We can have multiple reference variables bound to the same variable, but the other way is not possible. We cannot have a reference variable bound to more that one variable.

Example 9.3

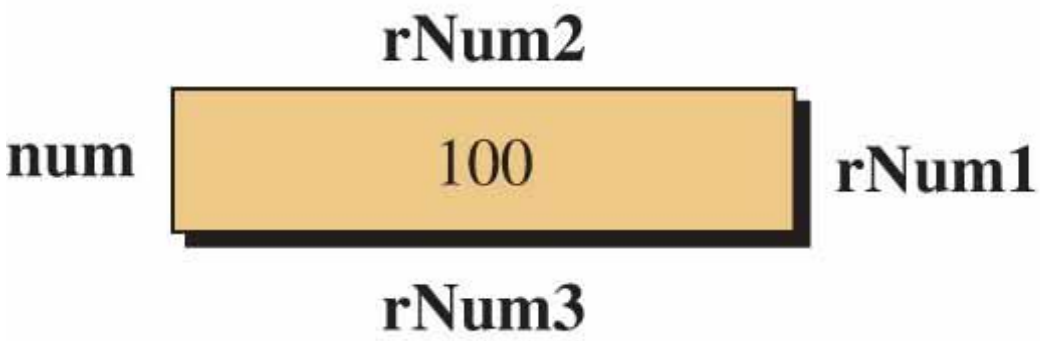
The following shows how to bind three reference variables to a single variable *num*.

```
int num = 100;
num& rNum1 = num;// rNum1 is bound to num
num& rNum2 = num;// rNum2 is bound to num
num& rNum3 = num;// rNum3 is bound to num
```


©2020 McGraw-Hill Education

Introduction Part 10

Figure 9.3 ***Multiple references to the same variable are allowed***



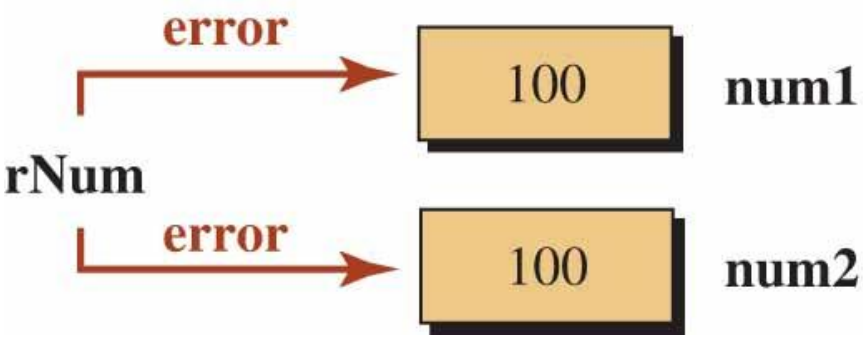
Note:

A location in memory has four names.

©2020 McGraw-Hill Education

Introduction Part 12

Figure 9.4 *A reference to multiple variables is not allowed*



Note:

rNum cannot be an alias name for two different locations, even if they hold the same value.

No Binding to Values.

Note that a reference variable cannot be bound to a value. For example, we get a compilation error in the following statement.

```
int& x = 92;  // Error: no binding to values
```


©2020 McGraw-Hill Education

Retrieving Values Part 1

When the reference relationship is established, the value stored in the common memory locations can be retrieved either through the original variable or reference variable.

Example 9.5

Program 9-1 shows the use of a original variable and its reference. However, we need to mention that this application is only for demonstration; the actual relationship between a variable and its reference is beneficial when we use them in two different functions as we will see later in the chapter.

©2020 McGraw-Hill Education

Retrieving Values Part 2

Program 9.1 *Accessing value*

```
/*********************************************************
 * The program shows how to declare and initialize the original*
 *and reference variables and then access the common value     *
 * through either of them.                                     *
 *********************************************************/
#include <iostream>
using namespace std;

int main ( )
{
    // Creation of reference relations
    int score = 92;
    int& rScore = score;
    // Using data variable
    cout << "Accessing value through data variable." << endl;
    cout << "score: " << score << endl;
    // Using reference variable
    cout << "Accessing value through reference variable." << endl;
    cout << "rScore: " << rScore;
    return 0;
```


©2020 McGraw-Hill Education

Retrieving Values Part 3

Program 9.1 *Accessing value*

```
}
```

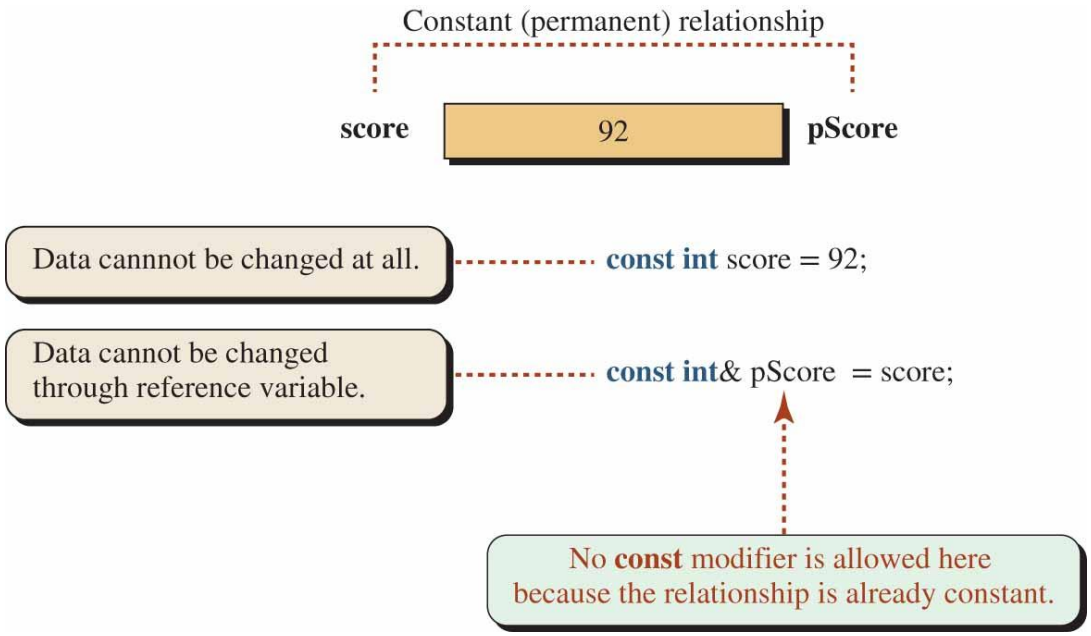
```
Run:
Accessing value through data variable.
score: 92
Accessing value through reference variable.
rScore: 92
```


©2020 McGraw-Hill Education

Changing Values

There is only one value in a reference relationship, but the value can be changed either through the original variable or any of the reference variables unless we use const modifiers.

Figure 9.5 ***Preventing change in a reference relation***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Changing Values Part 2 (1)

Table 9.1 ***Four possible combinations***

Case	Data Variable	Reference variable	Status
1	int name = value;	int& rName = name;	OK
2	const int name = value;	int& rName = name;	Error
3	int name = value;	const int& rName = name;	OK
4	const int name = value;	const int& rName = name;	OK

©2020 McGraw-Hill Education

Changing Values Part 2 (2)

First Case

In the first case, there is no restriction to changing the value either through the original variable or through the reference variable.

Second Case

The second case creates a compilation error because we try to bind a non-constant reference variable to a constant variable. Since the original variable is already constant, there is no way that we can change its value through the reference variable.

Third Case

In the third case, the data can be changed through the data variable, but we want to restrict it from being changed through the reference variable.

©2020 McGraw-Hill Education

Changing Values Part 3

Fourth Case

In the fourth case, we want to create an original variable and a reference variable in a way that neither the original variable nor the reference variable can change the common value. This case has little application because the data variable and the reference variables can only be used to retrieve data, not to change them.

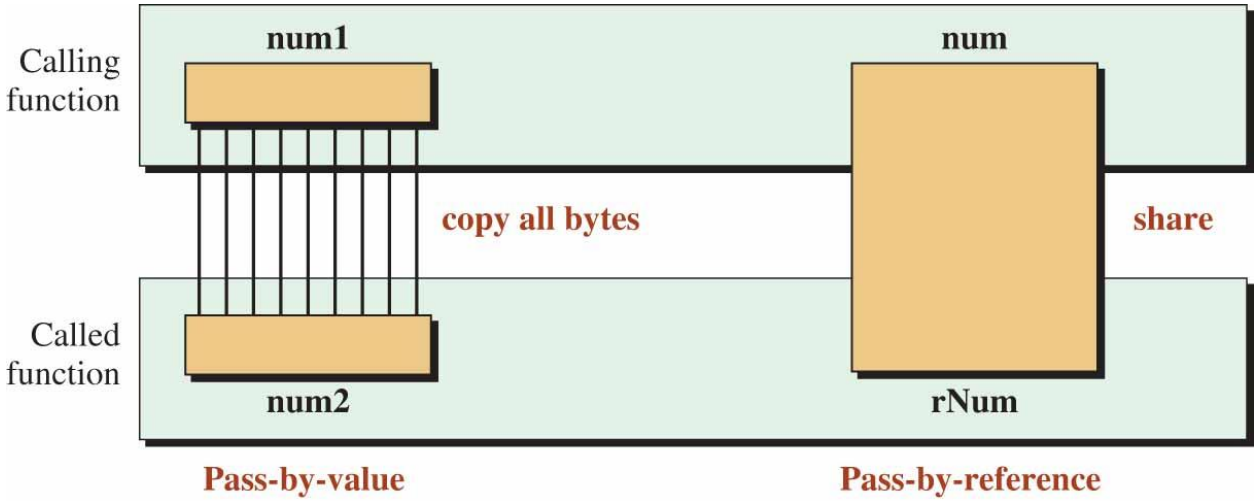
©2020 McGraw-Hill Education

Application Part I

Pass-By-Reference.

In the first case, a calling function has an object (or objects) that it needs to send to the called function for processing. We used what is called *pass-by-value* in the past; we want to show how to do it using *pass-by-reference*. Both methods are shown in Figure 9.6.

Figure 9.6 ***Comparing pass-by-value and pass-by-reference***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Applications Part 2 *(1)*

As a trivial example let us write two small programs, side-by-side, to compare the two methods. Assume we want to have a function that processes integers passed to it. The function can be designed as *pass-by-value* or *pass-by-reference*.

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int);

int main()
{
    int num = 10;
    doIt(num);
    return 0;
}
// Pass-by-value
void doIt(int num)
{
    // Code
}
```

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int&);

int main()
{
    int num = 10;
    doIt(num);
    return 0;
}
// Pass-by-reference
void doIt(int& rNum)
{
    // Code
}
```


©2020 McGraw-Hill Education

Applications Part 3

Characteristics of Pass-by-Value.

In the pass-by-value method, the calling function sends a copy of its argument to the called function.

The copy becomes the parameter in the called function. In other words, the following statement is done in the background:

```
int num2 = num1;
```

In pass-by-value, we have two independent objects: the argument and the parameter.

This means that changes in the parameter, intentional or accidental, cannot affect the argument.

This may be an advantage in one situation and a disadvantage in another situation.

©2020 McGraw-Hill Education

Applications Part 4

Another issue in using pass-by-value is the cost of copying. Copying all bytes of the argument can be costly if the object to be copied is large.

This means that if the object to be passed is of a fundamental type, we should not worry because the number of bytes to be copied is small (normally less that eight).

However, if we need to copy an object of class type with thousand of bytes, should consider other methods.

©2020 McGraw-Hill Education

Applications Part 5 (1)

Recommendation

After discussing the pros and cons of each method of passing an argument to a function, let us give a short recommendation to decide which method we should use.

1. **If we need to prevent change, we should use.**
   a. **pass-by-value for small objects.**
   b. **pass-by-constant reference for large objects.**
2. **If there is a need for change, we should use pass-by-reference.**

©2020 McGraw-Hill Education

Applications Part 6 (1)

Warning

We need to give a warning here. We cannot bind a reference parameter to a value argument. For example, the following code creates a compilation error.

```
void fun(int& rX) { … }  // Function definition
fun(5); // Function call (compilation error)
```

The parameter rX is a reference parameter. The argument of the function call needs to be a variable name, not a value.

©2020 McGraw-Hill Education

Applications Part 7 (1)

Example 9.6

Assume we want to write a function to print the value of a fundamental data type.

We do not want to change the value of the data in the calling function.

This is the first case in our recommendation; we can use either pass-by-value or pass-by-constant reference.

Since the object is small, we can use the pass-by-value.

©2020 McGraw-Hill Education

Applications Part 8

Example 9.7

Assume we want to write a copy constructor for a class.

This is also the first case in our recommendation because we do not want the function to change the original object.

However, pass-by-value is costly (the object can be large) and more important it is impossible because pass-by-value needs the call to the copy-constructor to copy the object, which means we need the copy constructor to create a copy constructor (vicious circle).

©2020 McGraw-Hill Education

Applications Part 9

The C++ standard says we must use pass-by-constant reference as shown below:

```
// Pass-by-reference in copy constructor
Circle :: Circle(const Circle& circle)
: radius(circle.radius)
{
}
```

Example 9.8

Assume we want to write a function to swap two data items. This is the second case in our recommendation.

We need change, so we use pass-by-reference as shown in Program 9-2.

©2020 McGraw-Hill Education

Applications Part 10 (1)

Program 9.2 *Using a swap function with pass-by-reference*

```
/*****************************************************
 * The program shows how to use pass-by-reference to allow a  *
 * called function to swap two values in the calling function.*
 *****************************************************/
#include <iostream>
using namespace std;

void swap (int& first, int& second) ; // Prototype

int main ( )
{
    // Definition of two variables
    int x = 10;
    int y = 20;
    // Printing the value of x and y before swapping
    cout << "Values of x and y before swapping." << endl;
    cout << "x: " << x << " " << "y: " << y << endl;
    // Calling swap function to swap the values of x and y
    swap (x , y);
    // Printing the value of x and y after swapping
```


©2020 McGraw-Hill Education

Applications Part 11 (1)

Program 9.2 *Using a swap function with pass-by-reference*

```
21      cout << "Values of x and y after swapping." << endl;
22      cout << "x: " << x << " " << "y: " << y;
23      return 0;
24  }
25  /*****************************************************************
26   * The swap function swaps the values of the parameters and     *
27   * pass-by-reference allows the corresponding arguments in main *
28   * to be swapped accordingly.                                   *
29   *****************************************************************/
30  void swap (int& rX, int& rY)
31  {
32      int temp = rX;
33      rX = rY;
34      rY = temp;
35  }
```

```
Run:
Values of x and y before swapping.
x: 10 y: 20
Values of x and y after swapping.
x: 20 y: 10
```


©2020 McGraw-Hill Education

Applications Part 12 (1)

Return-By-Reference.

In the second case, a called function has an object that needs to be returned to the calling function. We used return-by-value in the past; we want to show how to do it using return-by-reference.

©2020 McGraw-Hill Education

Applications Part 12 (2)

Characteristics of Return-by-Value.

This is the practice that we have used so far. The called function returns an object of the desired type using the following prototype.

```
type function(...);
```

Return-by-value is simple and can be used anywhere. We can return the value of a parameter or a local variable.

The only drawback is the cost of copying.

If the object to be returned is a fundamental type, we should not worry; if it is an object of a class type, we know that the copy constructor will be called and the cost may be high.

©2020 McGraw-Hill Education

Applications Part 13 (1)

Characteristics of Return-by-Reference.

In this practice, the type of the object to be returned is a reference to another object shown in the following prototype.

```
type & function(...);
```

This method eliminates the cost of copying, but it has a drawback.

We cannot return an object by reference if it is a value parameter or a local variable (except static).

The reason, as we will discuss later in the chapter, is that when a function is terminated all local variables and value parameters are destroyed.

When an object is destroyed we cannot have an alias name to it, but we can return a reference to a reference parameter.

©2020 McGraw-Hill Education

Applications Part 14 (1)

Example 9.9

Let us give a trivial example. Assume we want to write a function to find the larger between two integers.

We could use either the combination of pass-by-value and return-by-value or the combination of pass-by-reference and return-by-reference.

The following shows the two programs side-by-side for comparison.

©2020 McGraw-Hill Education

Applications Part 14 (2)

```
#include <iostream>
using namespace std;

// Return-by-value
int larger(int x, int y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```

```
#include <iostream>
using namespace std;

// Return-by-reference
int& larger(int& x, int& y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```


©2020 McGraw-Hill Education

Applications Part 15

Example 9.10

Let us now find the larger between two objects of class types. In a previous program we created a *fraction* class.

We do not repeat the interface and the implementation file here (separate compilation).

We just create an application file that uses pass-by-reference and return-by-reference to find the smaller of two fractions.

Program 9-3 shows the application file.

©2020 McGraw-Hill Education

Applications Part 16

Program 9.3 *Finding the larger of two fractions*

```
/*****************************************************************
 * The program creates two pairs of fractions and then calls a *
 * function named larger to find the larger in each pair       *
 *****************************************************************/
#include "fraction.h"

Fraction& larger (Fraction&, Fraction&); // Prototype

int main ( )
{
     // Creating first pair of fractions and finding the larger
     Fraction fract1 (3, 13);
     Fraction fract2 (5, 17);
     cout << "Larger of the first pair of fraction: " ;
     larger (fract1, fract2).print ();
     // Creating second pair of fractions and finding the larger
     Fraction fract3 (4, 9);
     Fraction fract4 (1, 6);
     cout << "Larger of the second pair of fractions: " ;
     larger (fract3, fract4).print ();
```


©2020 McGraw-Hill Education

Applications Part 17

Program 9.3 *Finding the larger of two fractions*

```
    return 0;
}
/*****************************************************************
 * The function gets two fractions by reference, compares them *
 * and returns the larger.                                     *
 *****************************************************************/
Fraction& larger (Fraction& fract1, Fraction& fract2)
{
    if (fract1.getNumer() * fract2.getDenom() >
                        fract2.getNumer() * fract1.getDenom())
    {
        return fract1;
    }
    return fract2;
}
```

Run:

```
Larger of first pair of fractions: 5/17
Larger of second pair of fractions: 4/9
```


©2020 McGraw-Hill Education

POINTERS

A *pointer* type is a compound type representing the address of a memory location.

A pointer variable is a variable whose contents are of pointer type.

In this section, we discuss addresses, pointer types, pointer variables and some related issues.

©2020 McGraw-Hill Education

Addresses Part 1

When we talk about addresses, we need to consider two separate concepts.

Addresses in memory and the address of a variable.

Addresses in Memory.

Computer memory is a sequence of bytes. When we say that a computer has a memory (RAM) of one kilobyte, it means that memory is made of 2^{10} or 1024 bytes.

Today's computers have memories in the range of megabytes (2^{20}), gigabytes (2^{30}), or even terabytes (2^{40}).

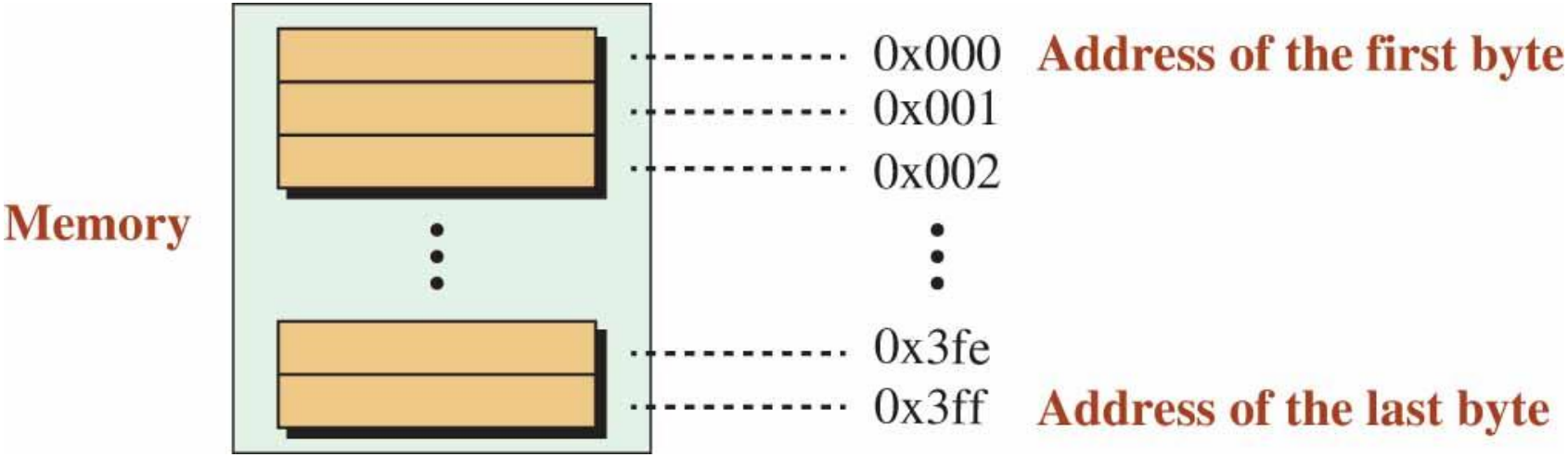
©2020 McGraw-Hill Education

Addresses Part 2

Each byte of memory has an address. The addresses are shown in hexadecimal format.

For example, in a computer with a kilobyte of memory, the bytes are numbered from 0x000 to 0x3ff (0 to 1023 in decimal).

Figure 9.7 ***Addresses in a RAM of one kilo byte***



©2020 McGraw-Hill Education

Addresses Part 3

Address of a Variable.

In our programs, we define variables of different types: Boolean, character, integer, floating-points, and classes.

Each variable occupies one or more bytes of memory. A variable of type *bool* or *char* normally occupies one byte of memory.

A variable of type *int* may occupy four or more bytes of memory.

A variable of type *double* may also occupy four or more bytes of memory (the *sizeof* operator can be used to determine the number of bytes).

©2020 McGraw-Hill Education

Addresses Part 4

The address of a variable is the address of the first byte it occupies.

This is an important point we need to remember when we are working with addresses or pointers as we will see.

To get the address of a variable, we use the *address operator* (&) in front of the variable.

The address of a variable is the address of the first byte occupied by that variable.

Example 9.11

Let us write a simple program to print the addresses of three variables: *bool*, *int*, and *double* (Program 9-4).

©2020 McGraw-Hill Education

Addresses Part 5

Program 9.4 *Printing size, value, and addresses*

```
/*****************************************************************
 * The program to define three variables and to print their    *
 * values and their addresses in the memory                    *
 *****************************************************************/
#include <iostream>
using namespace std;

int main ( )
{
     // Declaration of three data variables
     bool flag = true;
     int score = 92;
     double average = 82.56;
     // Printing size, value, and address of the flag variable
     cout << "A variable of type bool" << endl;
     cout << "Size: " << sizeof (flag) << " " ;
     cout << "Value: " << flag << " ";
     cout << "Address: "<< &flag << endl << endl;
     // Printing size, value, and address of the score variable
     cout << "A variable of type int" << endl;
```


©2020 McGraw-Hill Education

Addresses Part 6

Program 9.4 *Printing size, value, and addresses*

```
21      cout << "Size: " << sizeof (score) << " " ;
22      cout << "Value: " << score << " ";
23      cout << "Address: "<< &score << endl << endl;
24      // Printing size, value, and address of the average variable
25      cout << "A variable of type double" << endl;
26      cout << "Size: " << sizeof (average) << " " ;
27      cout << "Value: " << average << " ";
28      cout << "Address: "<< &average << endl;
29      return 0;
30  }
```

```
Run:
A variable of type bool
Size: 1 Value: 1 Address: 0x28fef0

A variable of type int
Size: 4 Value: 92 Address: 0x28fef1

A variable of type double
Size: 8 Value: 82.56 Address: 0x28fef5
```


©2020 McGraw-Hill Education

Addresses Part 7

Three important points in Program 9.4:

- **The extraction operator (<<) is overloaded to accept an address. Therefore, we can use the expression *cout << &score* to print the address of the variables.**
- **The addresses are displayed using hexadecimal notation. We need to be aware that the addresses are not integers, they are pointer types as we will see shortly.**
- **The sizes and addresses may be different in different systems. The addresses may start from the smallest to largest or vice versa. There may be some gaps in the address.**

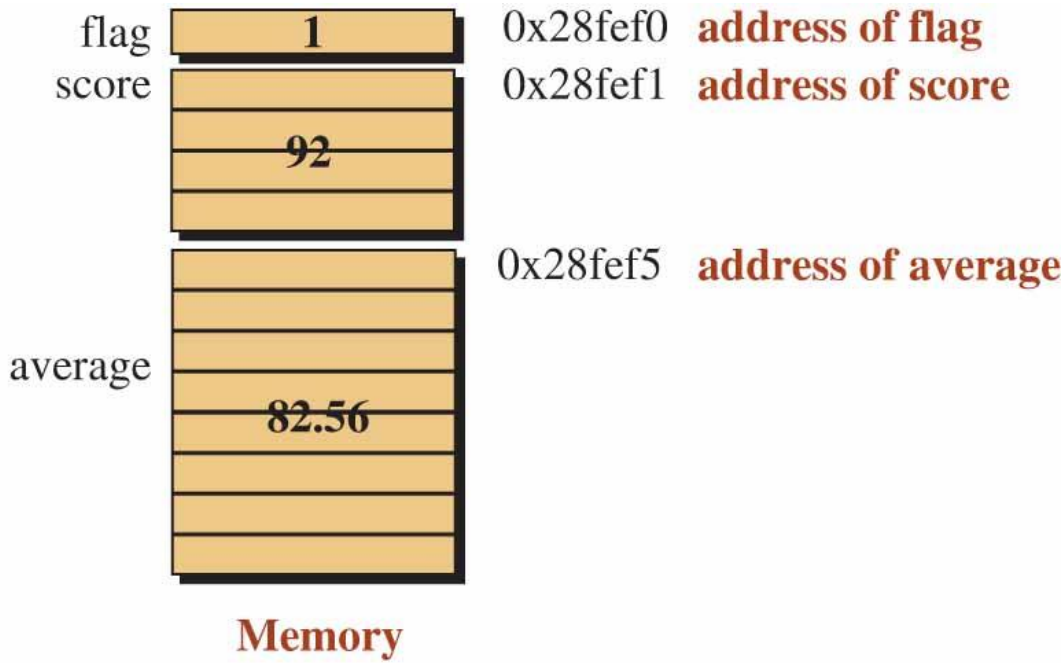
©2020 McGraw-Hill Education

Addresses Part 8

Figure 9.8 *Memory situation declared and defined*

Definition

```
bool flag = true;
int score = 92;
double average = 82.56;
```



©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 1

To allow us to manipulate addresses, C++ defines the pointer type and allows us to use pointer variables.

Pointer Type.

The pointer type is a compound type whose literal values are addresses.

It is a compound type in the sense that we have pointer to *char* pointer to *int*, pointer to *double*, and so on.

Note that we only talk about the address of a variable, not a value, because a value is always stored in a variable in memory.

To create a pointer to a type, we add the asterisk symbol after the type.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 2

The following shows some pointer types.

```
bool*       // A pointer to an object of bool type
int*     // A pointer to an object of int type
double*  // A pointer to an object of double type
Circle*  // A pointer to an object of Circle type
```

To create a pointer to a type, we add the asterisk symbol to the type.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 3

Pointer Variables.

We can store the value of pointer type in a pointer variable.

Although we can have different pointer types, what is stored in a pointer variable is a four-byte address.

In other words, the size of a pointer variable is fixed in C++, which means that a pointer variable can have an address from 0x00000000 to 0xFFFFFFFF.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 4

Declaration

To be able to use pointers, we need to have pointer variables.

To declare a pointer variable, we need to tell the computer that it is a pointer to a specific type.

The following shows some declaration examples.

```
bool*  pFlag;        // pFlag is variable of pointer to bool
int* pScore;         // pScore is variable of pointer to int
double* pAverage;    // pAverage is variable of pointer to double
```

Note that we read the declaration from right to left (*pFlag* is a pointer to a *bool*).

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 5

Initialization

A pointer variable, like a data variable, needs to be initialized before being used.

It needs to be initialized with a valid address in memory, which means that we cannot initialized it with a literal address; the address needs to be the address of an existing variable.

To do so, we use the address operator, the *ampersand*.

We will discuss this operator shortly, but the following shows how we can initialized our three declared pointer variables.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 6

```
// Initialize pFlag with address of flag
bool*  pFlag = &flag;
// Initialize pScore with address of score
int* pScore = &score;
// Initialize pAverage with address of average
double* pAverage = &average;
```

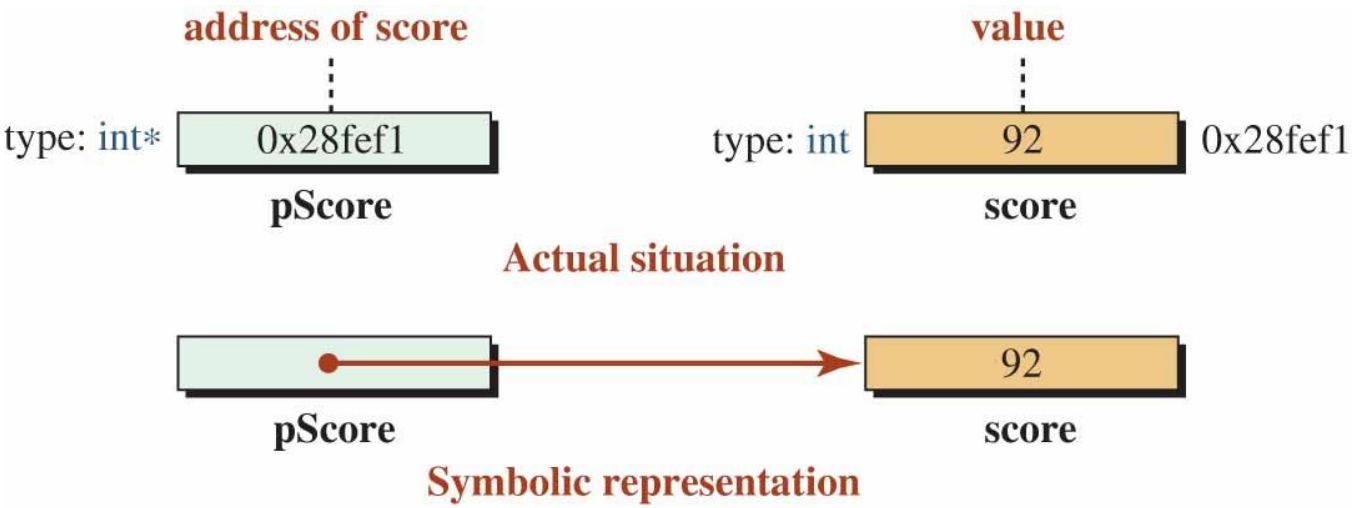
Note that pointer initialization has restrictions.

Since *pFlag* is a pointer to a variable of type *bool*, it needs to be initialized with the address of a variable of type *bool.*

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 7

Figure 9.9 *The pointed and the pointer variables*



Example 9.12

The following is a compilation error.

We cannot assign pointer literals to pointer variables.

```
double* pAverage = 0x123467; // Error. No literal addresses
```

Access the text alternative for slide images.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 8

Example 9.13

The following is a compilation error. We cannot assign pointer literals to pointer variables.

```
int num;
double* p = &num;    // Error. Variable p is of type double*
```

Indirection (dereference).

The whole purpose of declaration and initialization for a pointer variable is to use the address stored in the pointer variable to access the value stored in the pointed variable.

This is done through the use of the indirection operator (also called dereference operator) using the asterisk symbol as shown below:

```
*pFlage;   // value stored in flag
*pScore;   // value stored in score
*pAverage;    // value stored in average
```


©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 9

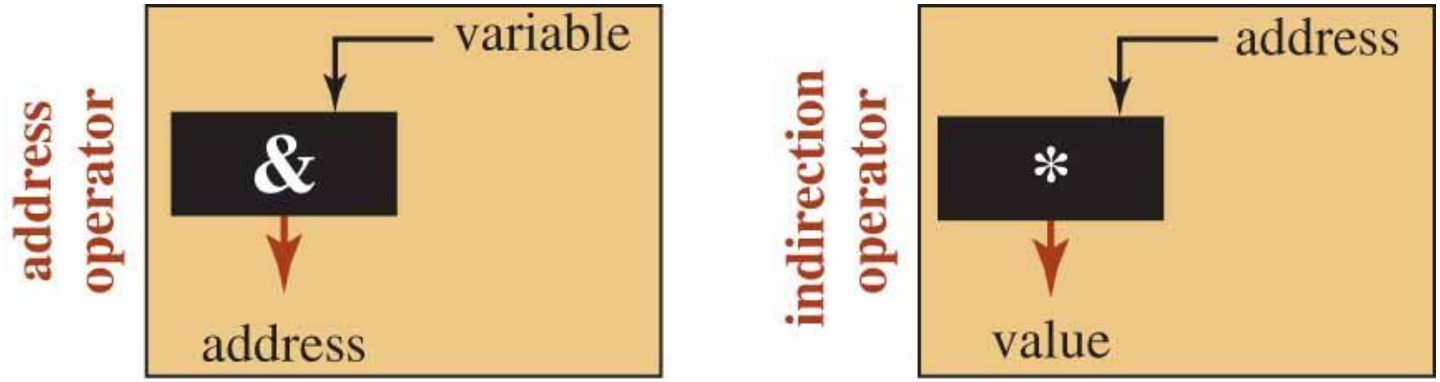
Two Related Operators.

The address-of and indirection operators are unary operators as shown in Table 9.2.

Table 9.2 The address expression

Group	Name	Operator	Expression	Pred	Assoc
Unary	address-of	&	&lvalue	17	←
	indirection	*	* pointer		

Figure 9.10 ***Address-of operator and indirection operator***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 10

C++ uses the ampersand and the asterisk symbols for three purposes as shown in Table 9.3.

***Table 9.3** Different use of ampersand and asterisk operator*

Symbol	type definition	Unary operator	Binary operator
&	type&	&variable	x & y
*	type*	*pointer	x * y

©2020 McGraw-Hill Education

Pointer Types and Pointer Variables Part 11

When the ampersand is used to define a reference type, it is put after the type (*int*&).

When it is used to get the address of the variable, it is put before the variable name (*&score*).

When it is used as a binary operator it is used between the two operands.

When the asterisk symbol is used to define a pointer type, it is put after the type (*int).**

When it is used to indirectly access a value, it is put before the pointer (pScore*).**

When it is used as a binary operator it is used between the two operands.

©2020 McGraw-Hill Education

Retrieving Value Part 1

After the pointer relationship is established, we can retrieve the value stored in the memory location either through the data variable or pointer variable.

Example 9.14

Program 9.5 is a simple program that accesses the values of a data variable directly and indirectly.

©2020 McGraw-Hill Education

Retrieving Value Part 2

Program 9.5 *Direct and indirect retrieval of data*

```
/*****************************************************************
 * The program shows how to access (retrieve or change) the value *
 * of a data variable both directly and indirectly.               *
 *****************************************************************/
#include <iostream>
using namespace std;

int main ( )
{
     // Declaration and initialization of variables
     int score = 92;
     int* pScore = &score;
     // Retrieving value of data variable directly and indirectly
     cout << "Direct retrieve of score: "<< score << endl;
     cout << "Indirect retrieve of score: "<< *pScore;
     return 0;
}
```

```
Run:
Direct retrieve of score: 92
Indirect retrieve of score 92
```


©2020 McGraw-Hill Education

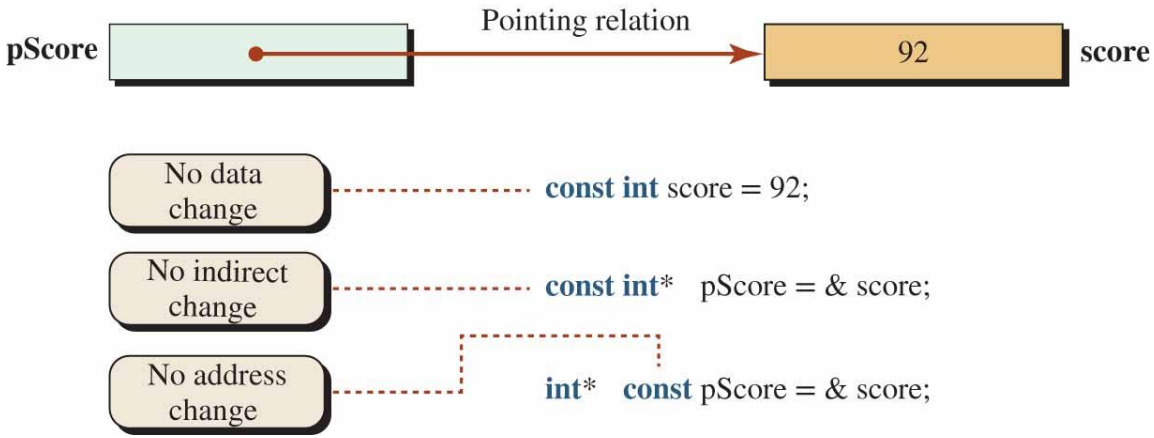
Using Constant Modifiers Part 1

Changes are controlled through the use of *const* modifiers.

We can use up to three *const* modifiers, each with a different purpose.

The first two are related to changing data; the third is related to changing the relationship between the data variable and the pointer variable.

Figure 9.11 ***Three uses of constant modifier***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Using Constant Modifiers Part 2

Controlling Data Change.

Table 9.4 shows the four possible combination that we can use to control data change.

Figure 9.4 ***Four possible combinations***

Case	Data Variable	Pointer variable
1	int name = value;	int* pName = &name;
2(forbidden)	const int name = value;	int* pName = &name;
3	int name = value;	const int& pName = &name;
4	const int name = value;	const int& pName= &name;

©2020 McGraw-Hill Education

Using Constant Modifiers Part 3

First Case

In the first case, there is no restriction to changing the value either through the data variable or through the pointer variable.

```
int score = 92;
int* pScore = &score;
score = 80;                // Change through data variable
*pScore = 70;              // Change through pointer variable
```


©2020 McGraw-Hill Education

Using Constant Modifiers Part 4

Second Case

The second case is a forbidden case. We get a compilation error when we try to bind the data variable to the pointer variable.

Since the data variable is already constant, there is no way that we can bind it to a non-constant pointer type.

```
const int score = 92;
int* pScore = &score;     // Compilation error
```


©2020 McGraw-Hill Education

Using Constant Modifiers Part 5

Third Case

In the third case, the data can be changed through the data variable, but we want to prevent it from being changed through the pointer variable.

If we try to change it through the pointer variable, we get compilation error.

```
int score = 92;
const int* pScore = &score;
score = 80;
*pScore = 80;    // Compilation Error
```


©2020 McGraw-Hill Education

Using Constant Modifiers Part 6

Fourth Case

In the fourth case, we want to create both a data variable and a pointer variable that cannot be changed.

This case has little application because the data variable and the pointer variables can only be used to retrieve data, not to change them.

```
const int score = 92;
const int* pScore = &score;
score = 80;   // Compilation Error
*pScore = 80;   // Compilation Error
```


©2020 McGraw-Hill Education

Using Constant Modifiers Part 7

Changing Pointer (Binding).

We can use the constant modifier to bind the data variable to the pointer variable for the lifetime of the program.

In other words, each of the four previous cases can be combined with either a non-constant or a constant pointer .

Figure 9.5 ***Eight possible combinations***

Changing Relationship between the data variable and the pointer variable	
any-of-four-previous-cases pName = &name;	// Changeable
any-of-four-previous-cases const pName = &name;	// Unchangeable

©2020 McGraw-Hill Education

Using Constant Modifiers Part 8

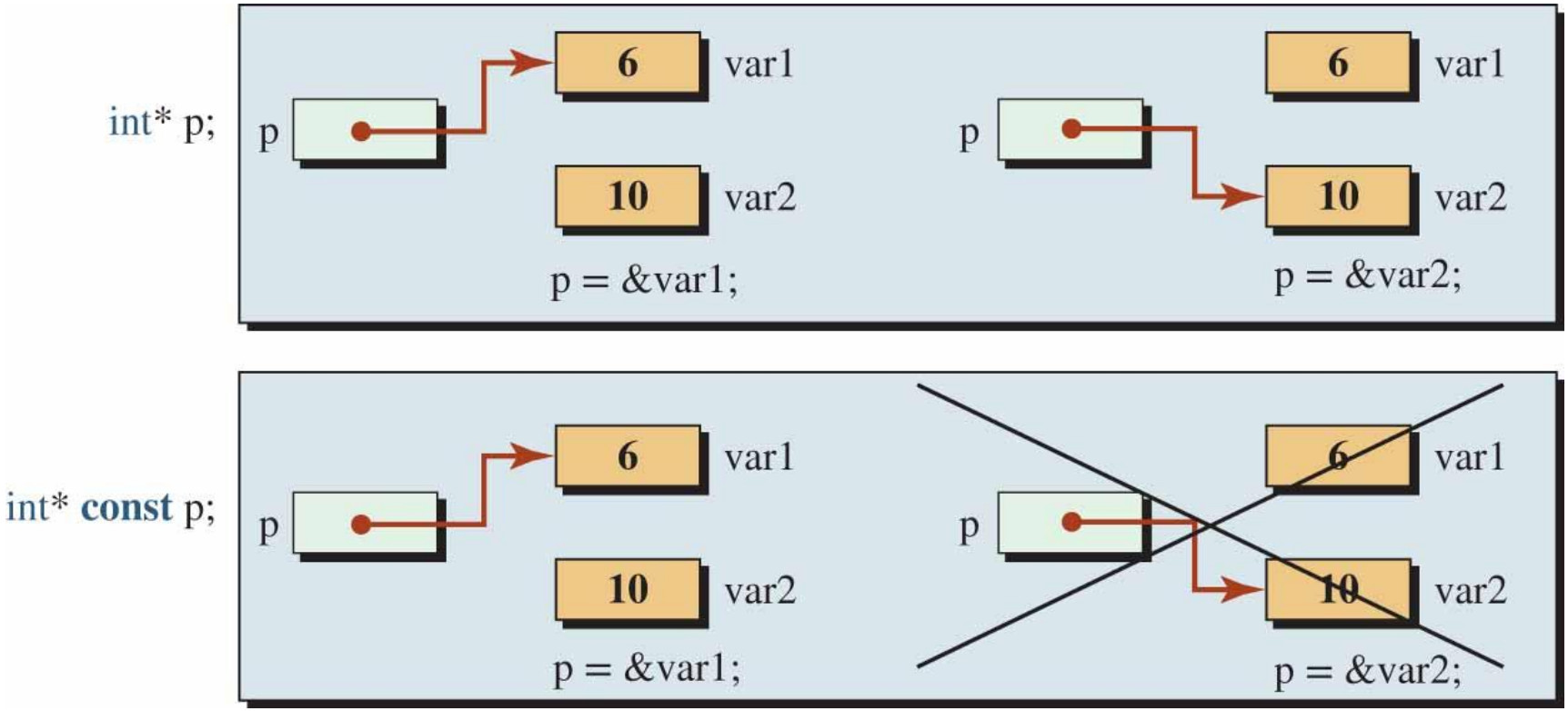
If the pointing relationship is constant, we cannot break the relationship and make the pointer point to another data variable.

Note that in this case, the *const* modifier needs to be put in front of the name of the pointer variable.

©2020 McGraw-Hill Education

Using Constant Modifiers Part 9

Figure 9.12 *Constant pointing relationship cannot be broken*



©2020 McGraw-Hill Education

Pointer to Pointer Part 1

A pointer variable is a location in memory.

We can store the address of this memory location in another memory location.

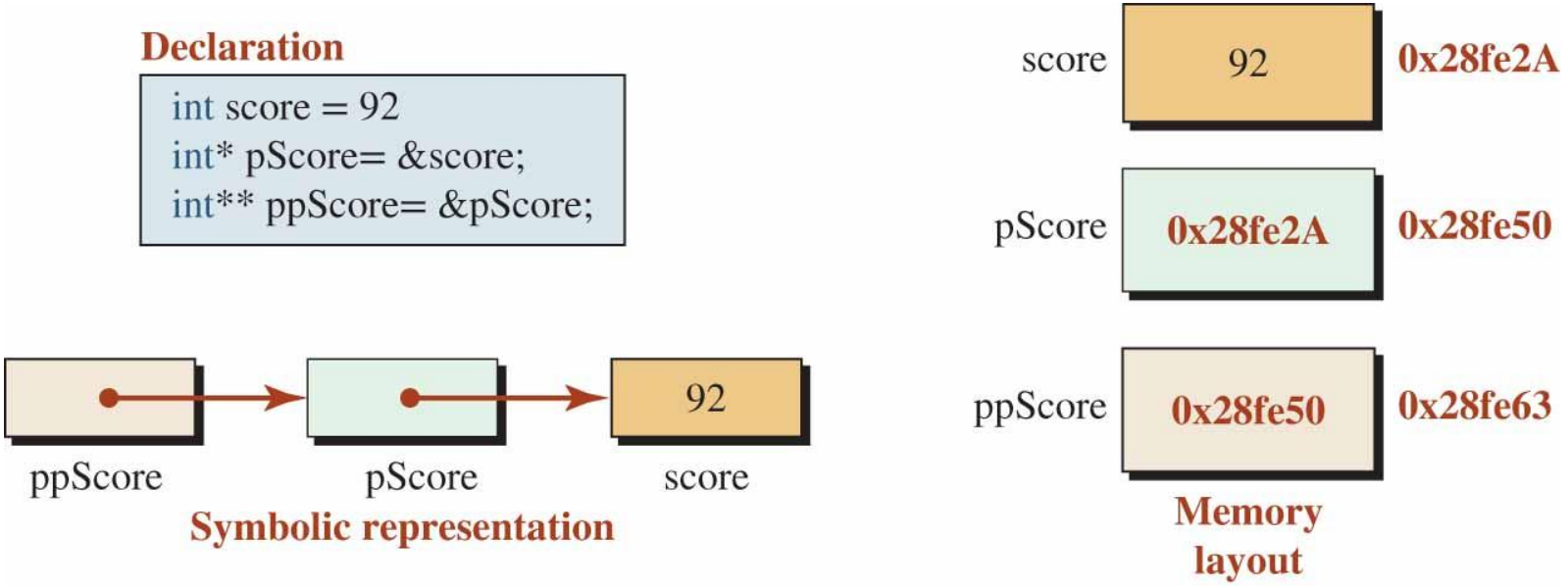
We can have new types: *pointer to pointer to int*, *pointer to pointer to double*, and so on.

```
int score = 92;                // Declaration and initialization
const int* pScore = &score;     // Binding pScore to score
int** pScore = &pscore;         // Binding ppScore to pscore
```


©2020 McGraw-Hill Education

Pointer to Pointer Part 2

Figure 9.13 *A pointer to pointer to int*



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Two Special Pointers Part 1

Sometimes we see two special pointers: pointer to nowhere and pointer to void.

We briefly explain this two pointers.

Pointer To Nowhere: Null Pointer.

A pointer to nowhere (sometimes called a null pointer) is a pointer that points to no memory location.

Although the C language used the word NULL to define such a pointer, the C++ language prefers using the literal 0.

In other words, if we want to show that a pointer is not pointing to any memory location at this moment, we can bind it to 0 as shown below.

```
int* p1 = 0;
double* p2 = 0;
```


©2020 McGraw-Hill Education

Two Special Pointers Part 2

Note that neither of the previous two statements means that p1 or p2 is pointing to the memory location at 0x0.

The byte at this address is used for the system and is not available to the user.

The literal value 0 simply means that the pointer at this moment is pointing to nowhere.

The programmer assigns 0 to a pointer to show that the pointer cannot be used until it is bound to a valid address.

Assigning 0 to a pointer variable means that declaration is still valid, but we cannot use it.

If a logic error does try use the pointer, the program aborts.

©2020 McGraw-Hill Education

Two Special Pointers Part 3

When a pointer is not null, its value can be interpreted as *true*; when it is null, its value can be interpreted as *false*.

```
int x = 7;
int* p = &x;
if (p) {…}     // Test is true here (p is not null)
p = 0;
if (p) {…}     // Test is false here (p is null)
```


©2020 McGraw-Hill Education

Two Special Pointers Part 4

Pointer To Void: Generic Pointer.

A *void pointer* is a generic pointer. It can point to an object of any type.

The following shows how we can use a generic pointer to point to any type.

```
void* p;            // Declaring a void pointer
int x = 10;
p = &x;                // Make p to point to an int type
double y = 23.4;
p = &y;             // Make p to point to a double type
```


©2020 McGraw-Hill Education

Applications

Just like references, pointers can be used to create communication between functions.

We discuss the use of pointers in passing data to a function and returning data from a function.

The first application is called pass-by-pointer; the second is called return-by-pointer.

Pass-By-Pointer.

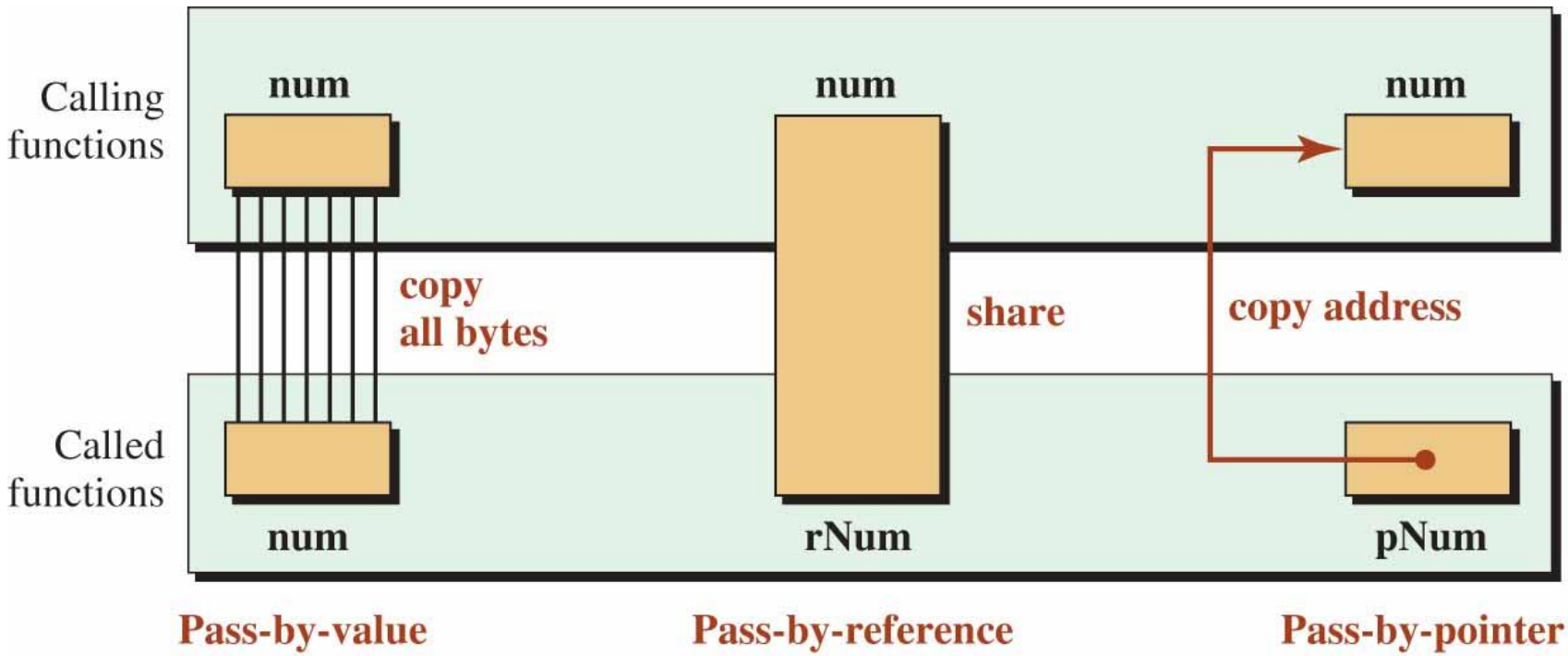
Pointers can be used to send data from a calling function to a called function.

We show pass-by-value, pass-by-reference, and pass-by-pointer.

©2020 McGraw-Hill Education

Applications Part 2 (2)

Figure 9.14 *Three methods of passing data to a function*



©2020 McGraw-Hill Education

Applications Part 3 *(3)*

As a trivial example let us write two small programs. Assume we want to have a function that does something to an integer passed to it. The function can be designed as pass-by-value or pass-by-pointer as shown below:

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int);

int main()
{
    int num1 = 10;
    doIt(num1);
    return 0;
}
// Pass-by-value
void doIt(int num2)
{
    // Code
}
```

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int*);

int main()
{
    int num = 10;
    doIt(&num);
    return 0;
}
// Pass-by-pointer
void doIt(int* pNum)
{
    // Code
}
```


©2020 McGraw-Hill Education

Applications Part 4 (1)

Characteristic of Pass-by-pointer.

In the pass-by-pointer method, the parameter is the address of the argument.

The binding between the two occurs in the background by the running environment as shown below.

```
int* pNum  = &num;
```


©2020 McGraw-Hill Education

Applications Part 4 (2)

In pass-by-pointer, the argument and the parameter are bound together.

Any change in the parameter means the same change in the argument unless we use a constant pointer as we discussed before.

It is obvious that pass-by-pointer reduces the cost of copying.

Only a four-byte address is copied. We need to consider this method when we want to pass a large object such as an object of class type to a called function.

©2020 McGraw-Hill Education

Applications Part 5 (2)

Recommendation

Let us augment the recommendation we made before to three practices:

pass-by-value,

pass-by-reference.

pass-by-pointer.

1. **If we need to prevent change, we must use.**
   a. **pass-by-value for small objects.**
   b. **pass-by-constant reference or pass-by-constant pointer for large objects.**
2. **If there is a need for change, we must use pass-by-reference or pass-by-pointer.**

©2020 McGraw-Hill Education

Applications Part 6 (2)

Example 9.15

Assume we want to write a function to print the value of a fundamental data type.

We definitely do not want to change the value of the data in the calling function.

This is the first case in our recommendation.

Since the object is small, we can use the pass-by-value. We have given several examples of this situation in the past.

Example 9.16

Assume we want to write a function to swap two data items. This is the second case in our recommendation.

We showed how to use pass-by-reference before; we show how to use pass-by-pointer in Program 9-6.

©2020 McGraw-Hill Education

Applications Part 7 (2)

Program 9.6 *Using a swap function with pass-by-pointer*

```
/*****************************************************************
 * The program shows how to use pass-by-pointer to allow a     *
 * called function to swap two values in the calling function. *
 *****************************************************************/
#include <iostream>
using namespace std;

void swap (int* first, int* second) ; // Prototype

int main ( )
{
     // Definition of two variables
     int x = 10;
     int y = 20;
     // Printing the value of x and y before swapping
     cout << "Values of x and y before swapping." << endl;
     cout << "x: " << x << " " << "y: " << y << endl;
     // Calling swap function to swap the values of x and y
     swap (&x , &y);
     // Printing the value of x and y after swapping
```


©2020 McGraw-Hill Education

Applications Part 8 (1)

Program 9.6 *Using a swap function with pass-by-pointer*

```
21      cout << "Values of x and y after swapping." << endl;
22      cout << "x: " << x << " " << "y: " << y;
23      return 0;
24  }
25  /*****************************************************************
26   * The swap function swaps the values of parameters and          *
27   * pass-by-pointer allows the corresponding arguments in main    *
28   * to be swapped accordingly.                                    *
29   *****************************************************************/
30  void swap (int* pX, int* pY)
31  {
32      int temp = *pX;
33      *pX= *pY;
34      *pY = temp;
35  }
```

```
Run:
Values of x and y before swapping.
x: 10 y: 20
Values of x and y after swapping.
x: 20 y: 10
```


©2020 McGraw-Hill Education

Applications Part 8 (2)

Return-By-Pointer.

In the second case, a called function has an object that needs to be returned to the calling function.

We can use what is called return-by-pointer.

Characteristics of Return-by-Pointer.

In this case, the type of the object to be returned is a pointer to a function parameter, which itself is a pointer to an object in the calling function as shown in the following prototype.

```
type*  function(type* ...);
```


©2020 McGraw-Hill Education

Applications Part 10 (2)

Example 9.17

Let us give a trivial example. Assume we want to write a function to find the larger of two integers.

We could use either the combination of pass-by-value and return-by-value or the combination of pass-by-pointer and return-by-pointer.

©2020 McGraw-Hill Education

Applications Part 11 (2)

```
#include <iostream>
using namespace std;

int larger(int x, int y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int* larger(int* x, int* y)
{
    if (*x > *y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = *larger(&x, &y);
    cout << z;
    return 0;
}
```


©2020 McGraw-Hill Education

Applications Part 12

Example 9.18

Let us now find the larger of two objects of class type. We used return-by-reference before; we repeat using return-by-pointer.

We just create an application file here that uses pass-by-pointer and return-by-pointer to find the smaller of two fractions.

Program 9-7 shows the application file.

Note that the member select expression, (*object –> member*), is in fact a short cut for the expression (object.member*).**

Both were discussed in a same previous chapter.

Also note that the object returned by a pointer is an lvalue, which means we can apply the member function print to the returned object as shown in lines 17 and 22.

©2020 McGraw-Hill Education

Applications Part 13 (2)

Program 9.7 *Finding the larger between two objects*

```
/*****************************************************************
 * The program creates two pairs of fractions and then calls a *
 * function named larger to find the larger in each pair.      *
 *****************************************************************/
#include "fraction.h"
#include <iostream>
using namespace std;

Fraction* larger (Fraction*, Fraction*); // Prototype

int main ( )
{
     // Creating the first pair of fractions
     Fraction fract1 (3, 13);
     Fraction fract2 (5, 17);
     cout << "Larger of the first pair of fraction: " ;
     larger (&fract1, &fract2) -> print ();
     // Creating the second pair of fractions
     Fraction fract3 (4, 9);
     Fraction fract4 (1, 6);
```


©2020 McGraw-Hill Education

Applications Part 14

Program 9.7 *Finding the larger between two objects*

```
21      cout << "Larger of the second pair of fractions: " ;
22      larger (&fract3, &fract4) -> print ();
23      return 0;
24  }
25  /*****************************************************************
26   * The function gets two fractions by pointer, compares them    *
27   * and returns the larger of the two.                           *
28   *****************************************************************/
29  Fraction* larger (Fraction* fract1, Fraction* fract2)
30  {
31      if (fract1 -> getNumer() * fract2 -> getDenom() >
32                      fract2 -> getNumer() * fract1 ->getDenom())
33      {
34          return fract1;
35      }
36      return fract2;
37  }
```

```
Run:
Larger of first pair of fractions: 5/17
Larger of second pair of fractions: 4/9
```


©2020 McGraw-Hill Education

ARRAY AND POINTERS

In C++, arrays and pointers are intertwined with each other.

Let us see how one-dimensional and two-dimensional arrays can be represented by pointers.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 1

When we declare an array named *arr* of type *t* and of capacity *N*, the system creates *N* memory locations of type *t*.

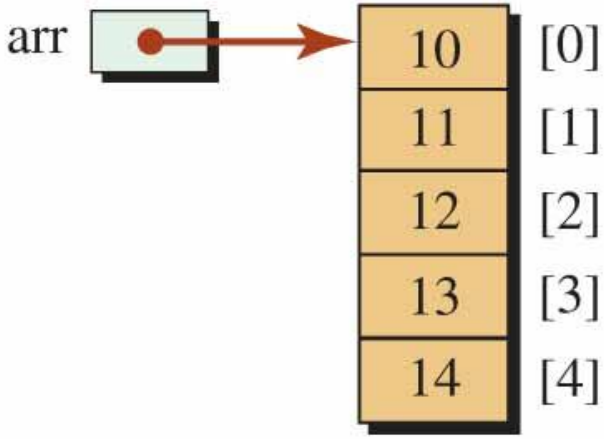
The system then creates a constant pointer of type *t* that is pointed to the first element.

Figure 9.15 ***Relationship between an array and a pointer***

```
int arr [5] = {10, 11, 12, 13, 14};
```

Declaration

Note:
The pointer *arr* is a constant pointer and cannot move.



Memory allocation

Access the text alternative for slide images.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 2

Since the address of the first element is fixed, we know the address of all the elements. The address of the element at index 0 is *arr* (or *arr* + 0); the address of the element at index 1 is *arr* + 1; and so on.

The name of the array is a constant pointer to the first element.

Program 9.8 is a small program that prints the addresses of the array elements defined in Figure 9.15 using the value of both the pointers and the address operator to prove that they are the same.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 3

Program 9.8 *A program to check the address of each array element*

```
/*****************************************************************
 * The program proves that the system stores the address of each *
 * element in the array in a constant pointer.                   *
 *****************************************************************/
#include <iostream>
using namespace std;

int main ()
{
     // Declaration of an array of five int
     int arr [5];
     // Printing the addresses through pointers and the & operator
     for (int i = 0; i < 5; i++)
     {
         cout << "Address of cell " << i << " Using pointer: ";
         cout << arr + i << endl;
         cout << "Address of cell " << i << " Using & operator: ";
         cout << &arr [i] << endl << endl;
     }
     return 0;
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 4

Program 9.8 *A program to check the address of each array element*

```
}
```

```
Run:
Address of cell 0 Using pointer: 0x28fee8
Address of cell 0 Using address operator: 0x28fee8

Address of cell 1 Using pointer: 0x28feec
Address of cell 1 Using address operator: 0x28feec

Address of cell 2 Using pointer: 0x28fef0
Address of cell 2 Using address operator: 0x28fef0

Address of cell 3 Using pointer: 0x28fef4
Address of cell 3 Using address operator: 0x28fef4

Address of cell 4 Using pointer: 0x28fef8
Address of cell 4 Using address operator: 0x28fef8
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 5

When referring to array elements using pointers, we need parentheses to give priority to the addition operator.

Example 9.19

Program 9-9 shows how we can refer to the elements of an array using the indexes and pointers with the same result.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 6

Program 9.9 *Using indexes and pointers with an array*

```
/*************************************************************
 * The program shows how to access the elements of an array q  *
 * using either the indexes or pointers to elements.          *
 *************************************************************/
#include <iostream>
using namespace std;

int main ( )
{
     // Declaration and initialization of an array
     int numbers [5] = {10, 11, 12,13, 14};
     // Accessing elements through the indexes
     cout << "Accessing elements through indexes" << endl;
     for (int i = 0; i < 5; i++)
     {
         cout << numbers [i] << " ";
     }
     cout << endl;
     // Accessing elements through the pointers
     cout << "Accessing elements through pointers" << endl;
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 7

Program 9.9 *Using indexes and pointers with an array*

```
21      for (int i = 0; i < 5; i++)
22      {
23          cout << *(numbers + i) << " ";
24      }
25      return 0;
26  }
```

```
Run:
Accessing elements through indexes
10    11    12    13    14
Accessing elements through pointers
10    11    12    13    14
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 8

Pointer Arithmetic.

We emphasize that pointer types are not integer types.

However, a limited number of arithmetic operators can be applied to pointer types.

Integer Addition or Subtraction.

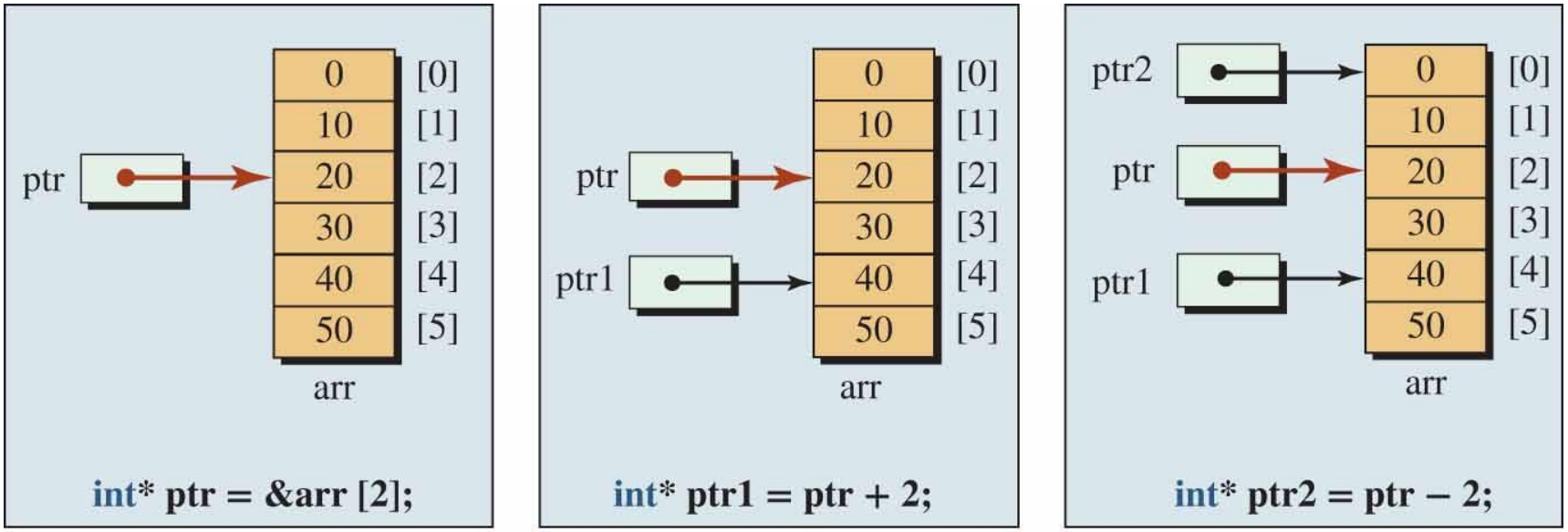
We can use the addition operator to add an integer to, or the subtraction operator to subtract an integer from a pointer:

```
Addition: ptr1 = ptr + n        Subtraction: ptr2 = ptr - n
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 9 (1)

Figure 9.16 *Integer addition and subtraction of pointers*



After creation of ptr | After creation of ptr1 | After creation of ptr2

Access the text alternative for slide images.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 9 (2)

Increment and Decrement.

These two operators are designed to be used with variables (lvalues).

They cannot be used with the name of the array because it is a constant value.

The meaning of *ptr*++ is the same as *ptr* = *ptr* + 1 and the meaning of *ptr*–– is the same as *ptr* = *ptr* – 1.

We should be aware that in these cases the original pointer moves.

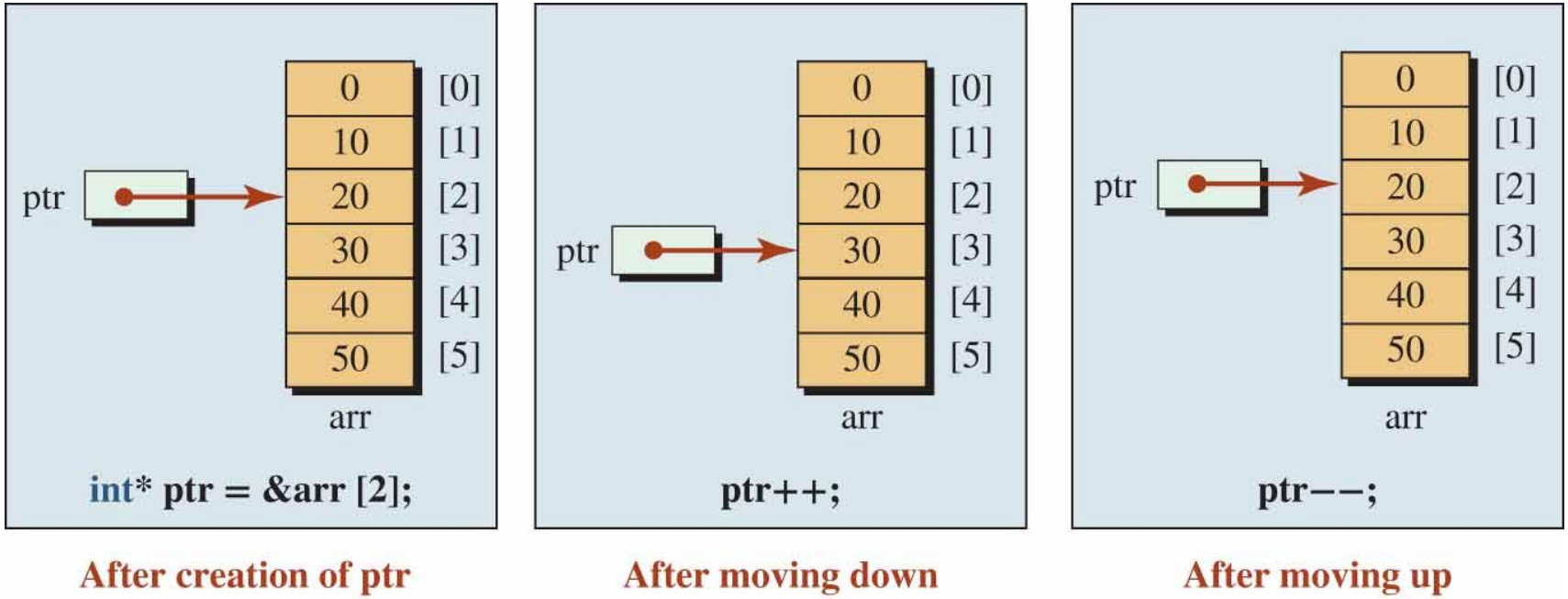
After the increment operation, the original pointer is pointing to the *arr* [3].

After the decrement operation, it is pointing back to *arr*[2].

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 10

Figure 9.17 *Increment and decrement operators with pointers*



After creation of ptr

After moving down

After moving up

Access the text alternative for slide images.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 11 (1)

Compound Addition and Subtraction.

We can also use compound addition and subtraction operators on pointer variables.

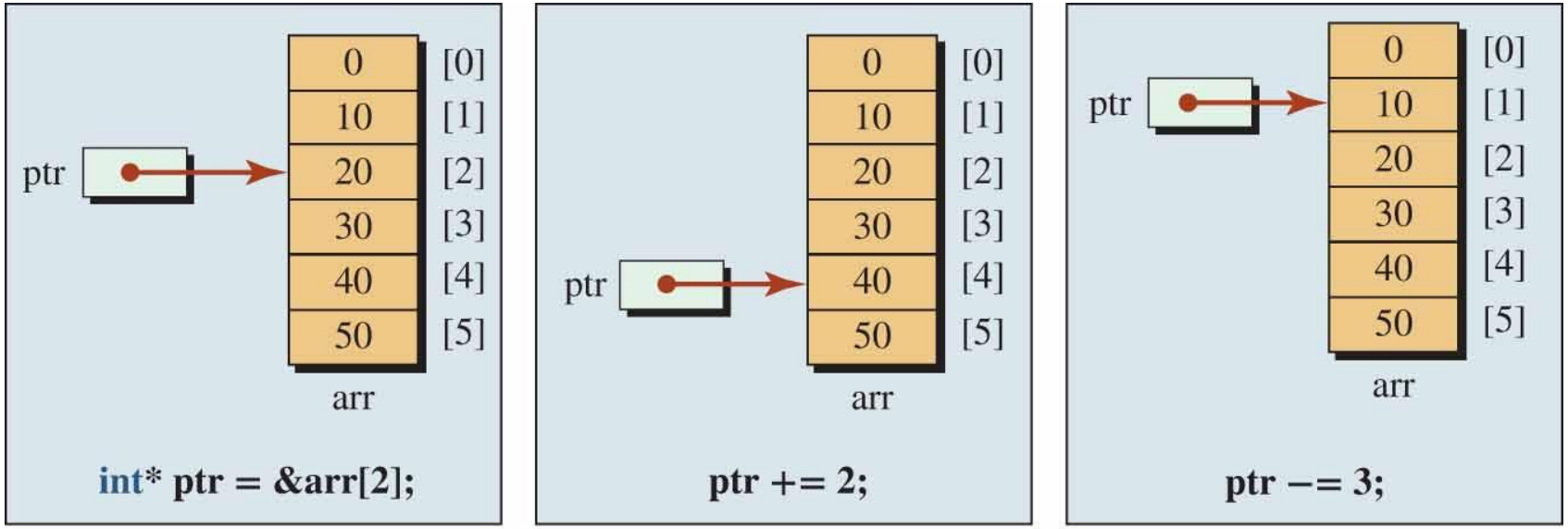
If *ptr* is a pointer variable, the expression (*ptr* += 3) is equivalent to (*ptr* = *ptr* + 3) and the expression (*ptr* −= 3) is equivalent to (*ptr* = *ptr* −3).

These operations, like the increment and decrement operators, move the pointer toward the beginning or toward the end.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 11 (2)

Figure 9.18 Compound addition and subtraction



After creation of ptr

After moving ptr down

After moving ptr up

Access the text alternative for slide images.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 12

Pointer Subtraction.

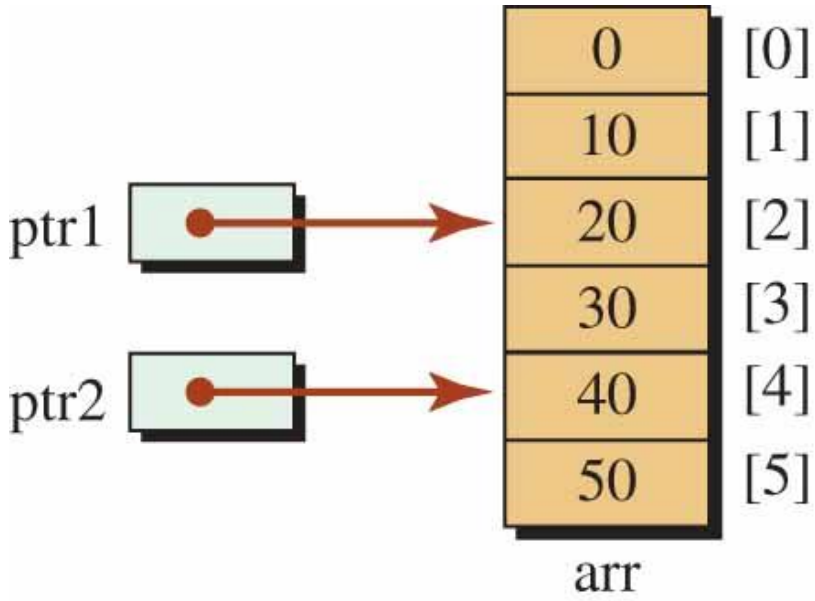
We can subtract one pointer from another.

If *ptr*1 and *ptr*2 are pointers in which *ptr*1 points to an element of a lower index and *ptr*2 to an element of a higher index, then the expression (*ptr*2 - *ptr*1) returns a positive integer and (*ptr*1 - *ptr*2) returns a negative integer.

In other words, the result of these operations are integers not pointers.

©2020 McGraw-Hill Education

Figure 9.19 *Pointer subtraction*




```
int n = ptr1 – ptr2 ;
```

n is 2

```
int m = ptr2 – ptr1 ;
```

m is –2

Access the text alternative for slide images.

©2020 McGraw-Hill Education

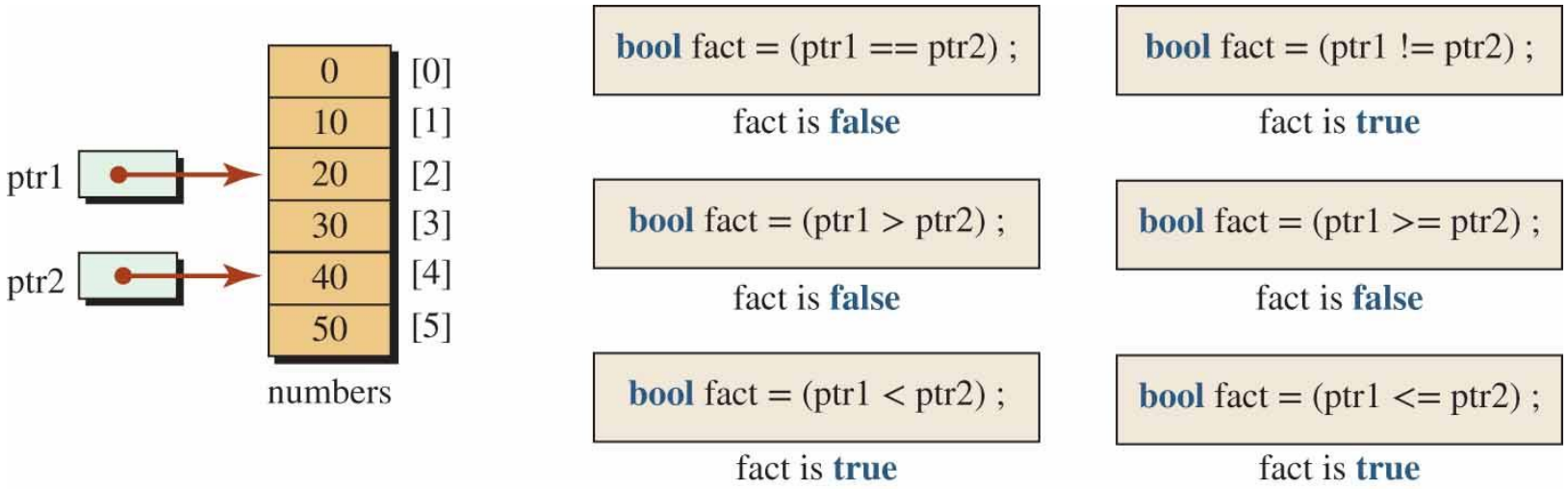
One-Dimensional Arrays and Pointers Part 13 (2)

No Pointer Addition.

The addition of pointer values are not allowed in C++ because it does not make sense.

We cannot add two pointers.

Figure 9.20 *Comparing Pointers*



Access the text alternative for slide images.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 14 (1)

No Other Operations.

We cannot use any other arithmetic operations on pointers.

For example, multiplication, division, and modulo operators are not defined for pointer types.

A Warning.

We need to use pointer arithmetic with caution.

If the operation causes access outside the array territory, we may destroy memory locations that are not part of the array.

©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 14 (2)

Functions With Array Parameters.

A pointer in a function can be used to represent an array.

Passing a Pointer to a Function for an Array.

One of the consequences of the above observation is that we can pass a pointer to a function instead of passing the array.

In other words, the following two prototypes are the same.

```
int getSum(const array [ ], int size);    // Using array
int getSum(const int* p, int size);       // Using pointer
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 15

Program 9.10 *Finding the sum of elements in an array*

```
/*****************************************************************
 * The program shows how to access the elements of an array    *
 * using pointers.                                              *
 *****************************************************************/
#include <iostream>
using namespace std;

int getSum (const int*, int); // Prototype

int main ()
{
     // Array declaration and initialization
     int arr [5] = {10, 11, 12, 13, 14};
     // Function call
     cout << "Sum of elements: " << getSum (arr, 5);
     return 0;
}
/*****************************************************************
 * The function gets a pointer to the first element of an      *
 * array and calculates and returns the sum of the elements.   *
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 16

Program 9.10 *Finding the sum of elements in an array*

```
 ***********************************************************/
int getSum (const int* p, int size)
{
    int sum = 0;
    for (int i = 0; i < size ; i++)
    {
        sum += *(p++);
    }
    return sum;
}
```

Run:

```
Sum of elements: 60
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 17

Program 9.11 *Reversing the elements of an array*

```
/*****************************************************************
 * The program shows how a function can reverse the elements   *
 * of an array using a pointer.                                *
 *****************************************************************/
#include <iostream>
using namespace std;

void reverse (int* , int );

int main ()
{
    // Array declaration and initialization
    int arr [5] = {10, 11, 12, 13, 14};
    // Calling function
    reverse (arr, 5);
    // Printing array after reversed
    cout << "Reversed array: ";
    for (int i = 0; i < 5; i++)
    {
        cout << *(arr + i) << " ";
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 18

Program 9.11 *Reversing the elements of an array*

```
    }
    return 0;
}
/*****************************************************************
 * The function uses a pointer to the first element of the array *
 * and the size of the array to reverse the elements in place.   *
 *****************************************************************/
void reverse (int* pArr, int size)
{
    int i = 0;
    int j = size - 1;
    while (i < size / 2)
    {
        int temp = *(pArr + i);
        *(pArr + i) = *(pArr + j);
        *(pArr + j) = temp;
        i++;
        j- -;
    }
}
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 19

Program 9.11 *Reversing the elements of an array*

```
Run:
Reversed array: 14 13 12 11 10
```


©2020 McGraw-Hill Education

One-Dimensional Arrays and Pointers Part 20

Returning An Array from a Function.

Although we may think about returning an array from a function, we need to remember that an array is a combination of two pieces of information: the pointer to the first element and the size of the array.

A function can return only one piece of information (unless we bundle the pointer and the size in an object).

This is a reason that we mentioned in a previous chapter that we cannot return an array from a function.

©2020 McGraw-Hill Education

Two-Dimensional Arrays and Pointers Part 1

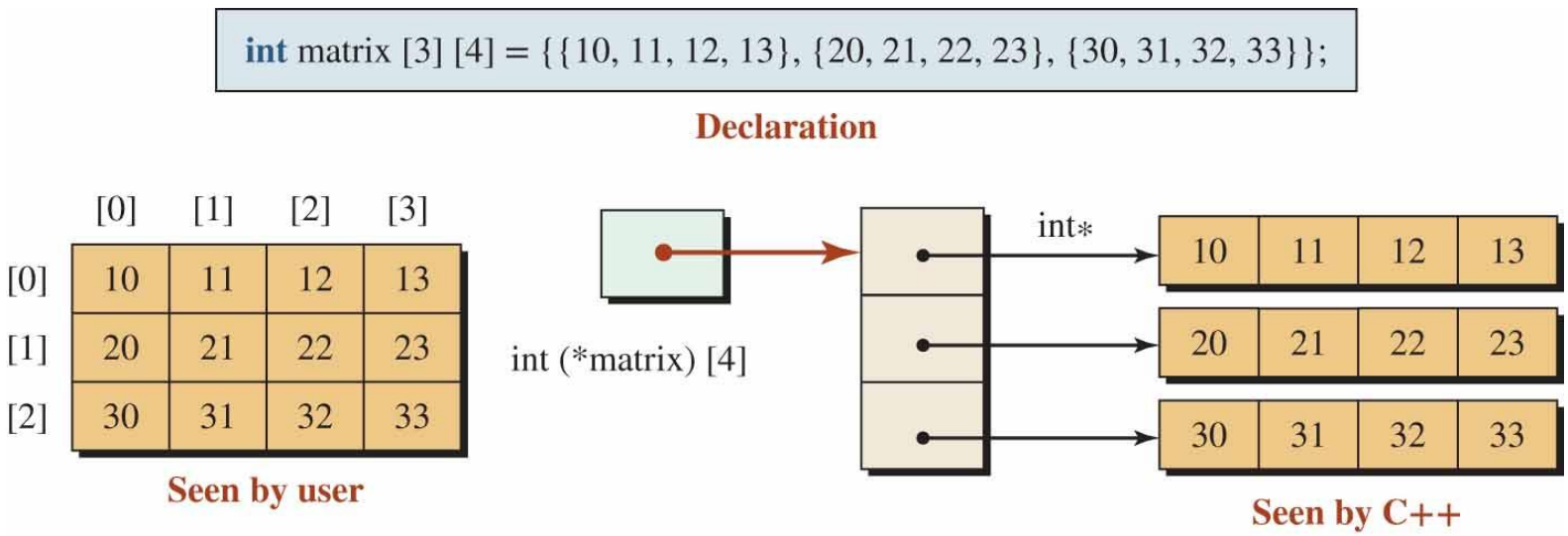
A two-dimensional array in C++ is an array of arrays.

If we keep this fact in mind, then it is easy to use pointers with two dimensional arrays.

©2020 McGraw-Hill Education

Two-Dimensional Arrays and Pointers Part 2

Figure 9.21 *A two-dimensional array seen by user and C++*



Contrary to some belief, the name of a two-dimensional array is not a pointer to pointer to integer; it is a pointer to an array of four integers or *int* (*matrix) [4].**

©2020 McGraw-Hill Education

Two-Dimensional Arrays and Pointers Part 3

Program 9.12 *Passing a two-dimensional array to a function*

```
/*****************************************************************
 * The program to show how to pass a two-dimensional array to a *
 * function using pointer notations.                            *
 *****************************************************************/
#include <iostream>
using namespace std;

void print (int (*) [4], int); // Prototype

int main ()
{
    int matrix [3][4] = {{10, 11, 12, 13}, {20, 21, 22, 23},
                                           {31, 32, 33, 34}};
    // Calling print function
    print (matrix, 3);
    return 0;
}
/*****************************************************************
 * The function accepts a pointer to any array of four integers *
 * with the number of rows.                                     *
```


©2020 McGraw-Hill Education

Two-Dimensional Arrays and Pointers Part 4

Program 9.12 *Passing a two-dimensional array to a function*

```
 ***************************************************************/
void print (int (*m) [4], int rows)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0 ; j < 4; j++)
        {
            cout << m[i][j] << " ";
        }
        cout << endl;
    }
}
```

Run:

```
10    11    12    13
20    21    22    23
31    32    33    34
```

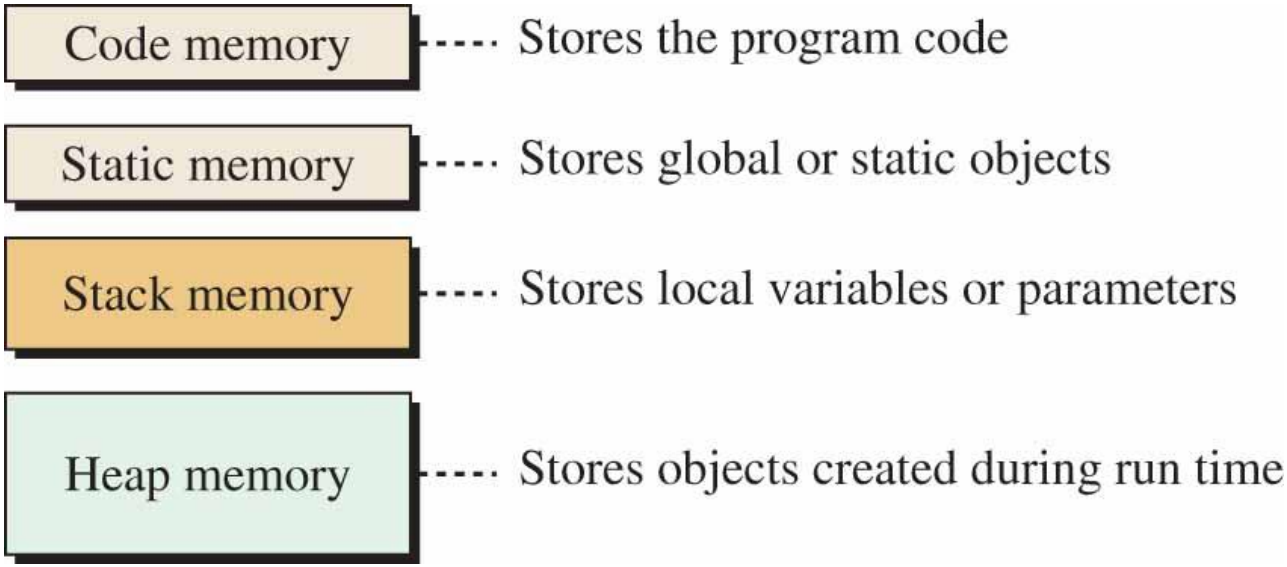

©2020 McGraw-Hill Education

MEMORY MANAGEMENT

When a program in C++ is running, it uses memory locations.

The C++ environment divides memory into different areas.

Figure 9.22 ***Memory sections used by a program***



©2020 McGraw-Hill Education

Code Memory

Code memory holds the program code.

When the program is running, the running environment of C++ executes each statements one after another or branches to another statement.

No objects are stored in this area of memory.

This part of memory is released when the program is terminated.

Code memory stores the program code;
it is released when the program terminates.

©2020 McGraw-Hill Education

Static Memory Part 1

Static memory is used to hold global objects (those that are not part of any function including *main*) and static objects created anywhere in the program (in the global area or inside functions).

These objects are automatically destroyed and their memory is released when the program is terminated.

Static memory stores the global and static objects; it is released when the program terminates.

©2020 McGraw-Hill Education

Static Memory Part 2

Program 9.13 *Using static memory*

```
/*****************************************************************
 * The program shows how global and static objects are visible *
 * any point in the program. They last through the life of the *
 * the program.                                                *
 *****************************************************************/
#include <iostream>
using namespace std;

int first = 20; // Global variable in static memory
static int second = 30 ; // Static variable created in static memory

int main ()
{
     static int third = 50; // Static variable in static memory

     cout << "Value of Global variable: " << first << endl;
     cout << "Value of Global static variable: " << second << endl;
     cout << "Value of local static variable: " << third;
     return 0;
}
```


©2020 McGraw-Hill Education

Static Memory Part 3

Program 9.13 *Using static memory*

```
Run:
Value of Global variable: 20
Value of Global static variable: 30
Value of local static variable: 501.16
```


©2020 McGraw-Hill Education

Stack Memory Part 1

The part of memory used by a program to hold local or parameter objects of functions is *stack memory*.

A stack is a last-in, first-out container. Whatever is pushed last, it is popped first.

This characteristic of a stack is well suited for storing parameters and local variables in functions.

Figure 9.23 shows the functions calls and the behavior of stack memory for a simple program that calls the function first, which in turn calls the function second.

©2020 McGraw-Hill Education

Stack Memory Part 2

We have shown pushing at the left-hand side and popping at the right-hand side.

The stack memory is empty at the beginning.

When the running environment calls the *main* function, it pushes its only local variable (x) into the stack (*main* has no parameter).

When *main* calls the function first, the system pushes its only parameter (a) into the stack.

When the function first calls second, the system pushes its only parameter (b) into the stack.

©2020 McGraw-Hill Education

Stack Memory Part 3

When the function second returns, the system pops its only parameter (b) and throws it away.

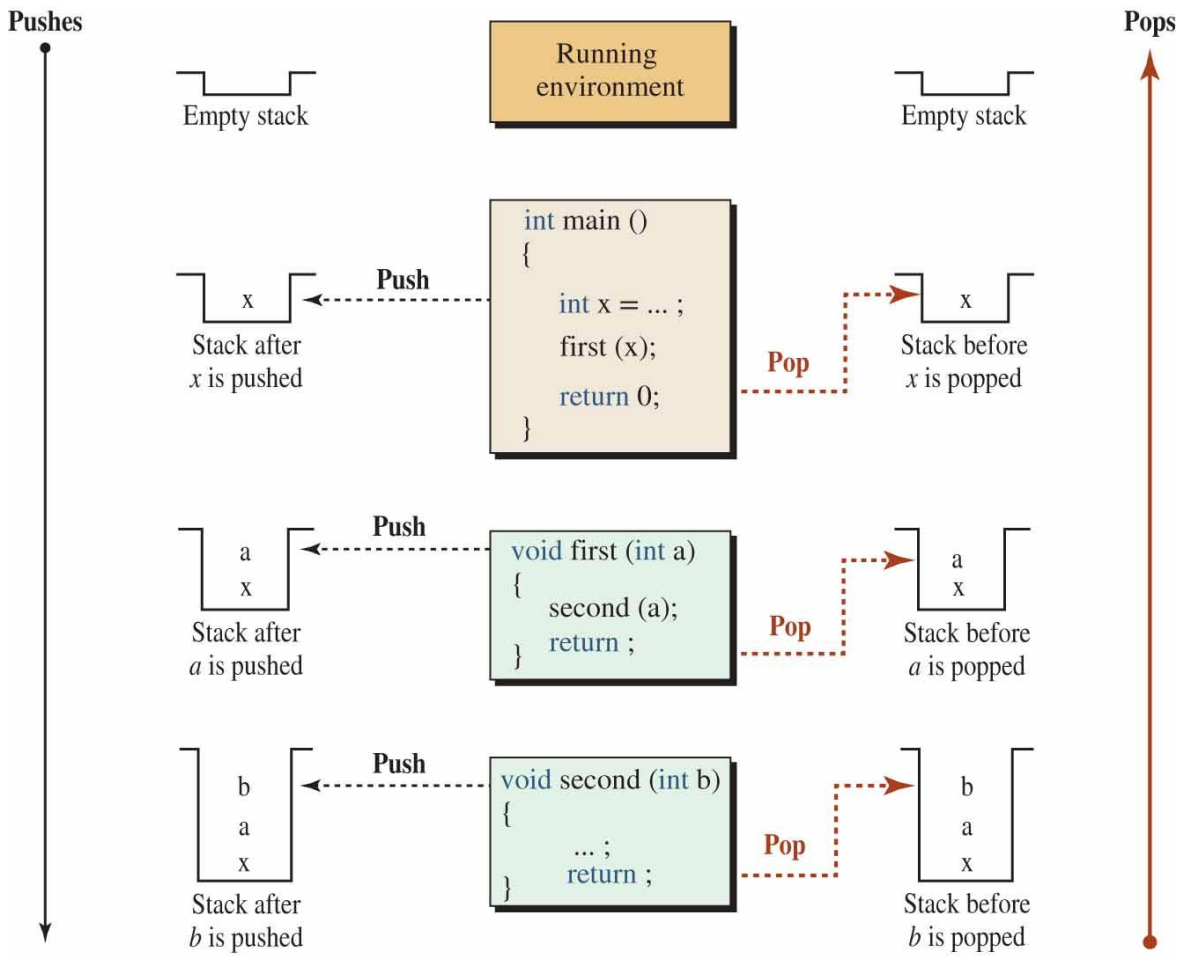
When the function first returns, the system pops its only parameter (a) and throws it away.

When the function *main* returns, the system pops its only local variable (x) and throws it away.

©2020 McGraw-Hill Education

Stack Memory Part 4

Figure 9.23 *Operation of stack memory*



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Stack Memory Part 6

Advantage

We have used stack memory in every program.

The last-in first-out operation of stack memory makes it very efficient.

Object are stored and remained in stack memory only when they are in a function's scope.

When an object goes out of the function's scope, it is popped and thrown away; it is not accessible any more.

©2020 McGraw-Hill Education

Stack Memory Part 7

Limitations

The efficiency of using stack memory creates two restrictions on its use:

- **The objects need to have a name at compilation time. No unnamed object can be stored in a stack.**
- **The size of the object needs to be defined at compilation time.**

  The system cannot allocate stack memory for an object unless it knows the exact size of the object.

  For a single object, the size can be inferred from the type; for a list of objects, the number of elements needs to be defined at compilation time.

©2020 McGraw-Hill Education

Stack Memory Part 8

An object created in stack memory needs to be given a name and have a size during compilation.

Example 9.24

Assume that we need a variable-size array, which means that every time we run the program, the array size needs to be defined by the user as shown below:

```
#include <iostream>
using namespace std;

int main ()
{
    int size;
    cin >> size;
    double array [size];        // Compilation error
    ...
    return 0;
}
```


©2020 McGraw-Hill Education

Heap Memory Part 1

There are occasions when we need to create objects in memory during run-time.

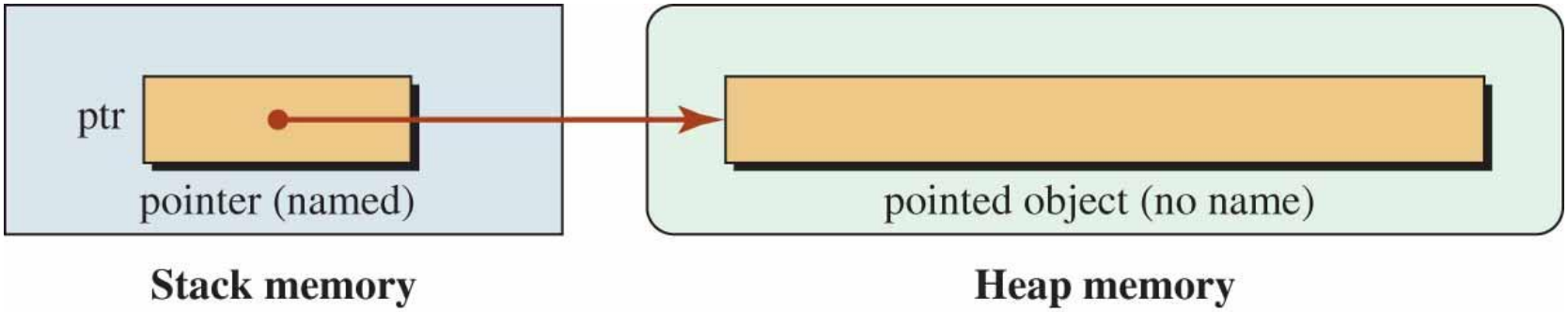
Heap memory (also called free memory or dynamic memory) is used to store objects created during runtime.

The objects created in heap memory cannot have a name, so to access them we need a pointer in stack memory that can be pointed to them.

©2020 McGraw-Hill Education

Heap Memory Part 2

Figure 9.24 *A pointer in the stack and object in the heap*



Note:
The pointer is in stack memory; it must have a name (*ptr*).
The pointed object is in heap memory; it cannot have a name.

Access the text alternative for slide images.

©2020 McGraw-Hill Education

Heap Memory Part 3

An object created in heap memory cannot have a name; it can be accessed only through its address reached by a pointer in stack memory.

Two Operators: new and delete.

Now the question is how we can create an object in the heap during runtime, and how we can destroy it when it is not needed, if the object has no name.

This is done through four operators.

Table 9.6 The new and delete operators

Group	Name	Operator	Expression	Pred	Assoc
Unary	Allocate object	new	new type	17	←
	allocate array	new [ ]	new type [size]		
	delete object	delete	delete ptr		
	delete array	delete [ ]	delete [ ] ptr		

©2020 McGraw-Hill Education

Heap Memory Part 4 (1)

The first operator is used to create memory in the heap for a single object.

The second operator is used to create an array of objects in the heap.

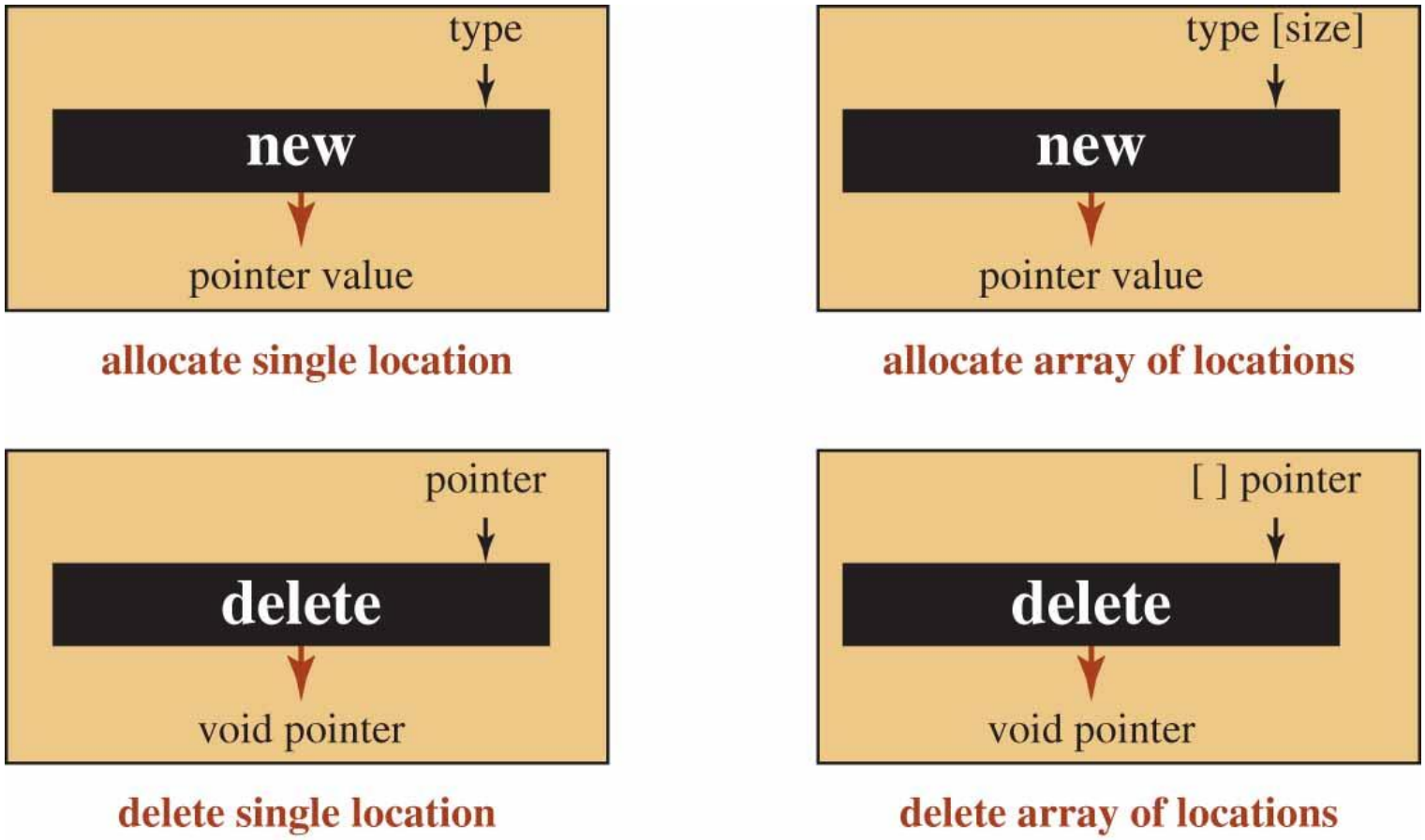
The third operator is used to delete the single object using its pointer.

The fourth operator is used to delete allocated memory for an array in the heap.

©2020 McGraw-Hill Education

Heap Memory Part 4 (2)

Figure 9.25 *The new and delete operators*



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Heap Memory Part 6

Example 9.25

Let us have a trivial example to create an object in the heap.

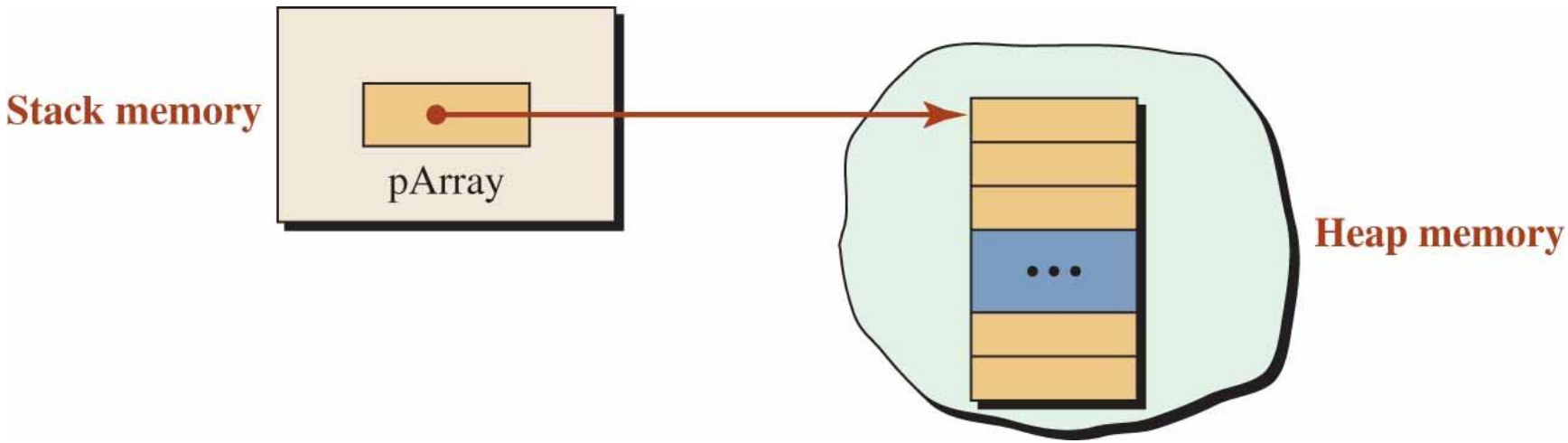
Assume we want to write a program that creates a variable-size array each time the user runs the program.

Definitely, this array cannot be created in stack memory because the size of the array is not defined during compilation time; it is only defined during runtime.

©2020 McGraw-Hill Education

Heap Memory Part 7

Figure 9.26 *A pointer in the stack and an array in the heap*



©2020 McGraw-Hill Education

Heap Memory Part 8

Program 9.14 *Using heap memory to store a variable-size array*

```
/*************************************************************
 * The program shows how to create and access a variable-size  *
 * array in the heap and use pointers to access elements.      *
 *************************************************************/
#include <iostream>
using namespace std;

int main ( )
{
    // Declaration of array size and the pointer in the stack
    int size;
    int* pArray;
    // Validation of the size to be greater than zero
    do
    {
        cout << "Enter the array size (larger than zero): ";
        cin >> size;
    } while (size <= 0);
    // Creation of array in the heap
    pArray = new int [size];
```


©2020 McGraw-Hill Education

Heap Memory Part 9

Program 9.14 *Using heap memory to store a variable-size array*

```
    // Inputting the values of array
    for (int i = 0; i < size ; i++)
    {
        cout << "Enter the value for element " << i << ": ";
        cin >> *(pArray + i);
    }
    // Outputting the values of the array
    cout << "The elements in the array are: " << endl;
    for (int i = 0; i < size ; i++)
    {
        cout << *(pArray + i) << " ";
    }
    // Deleting the array in the heap
    delete [ ] pArray;
    return 0;
}
```


©2020 McGraw-Hill Education

Heap Memory Part 10

Program 9.14 *Using heap memory to store a variable-size array*

```
Run:
Enter the array size (larger than zero): 3
Enter the value for element 0: 6
Enter the value for element 1: 12
Enter the value for element 2: 5
The elements in the array are:
6   12   5
```


©2020 McGraw-Hill Education

Heap Memory Part 11

Issues Related to Heap Memory.

When we work with heap memory and use the related operators (new and delete), we need to be aware of some problems that may occur.

Deleting without Allocating.

One of the errors that may occur in programming is that we try to use the *delete* operator without using the *new* operator first.

This means we try to delete an object in the heap without allocating it.

```
double x = 23.4;
double* pX = &x;     // Allocation in the stack
delete pX;           // Deletion in the heap may create run-time error
```


©2020 McGraw-Hill Education

Heap Memory Part 12

Allocating without Deleting (Memory Leak).

A more serious problem occurs when we allocate an object in the heap but we do not delete it as shown below:

```
double* pX = new double;
…  // using the allocated memory
```


©2020 McGraw-Hill Education

Heap Memory Part 13

Dangling Pointer.

Another problem that may occur is when we delete the pointed object and then try to use it again as shown below.

```
double* pX = new double;
…           // using the allocated memory
delete pX;
*pX = 35.3; // Dangling pointer
```

When we work with the heap memory, we need to pair each new operator with the delete operator and avoid dangling pointers.

Recommendation.

Our strong recommendation is always explicitly delete any memory locations created in the heap.

©2020 McGraw-Hill Education

Two-Dimensional Array Part 1

A two-dimensional array is made of rows and columns.

We have three choices to create a two-dimensional array.

Using Only Stack Memory.

If both the row size and the column size are known before compilation, we can create the array totally in the stack as we have done before.

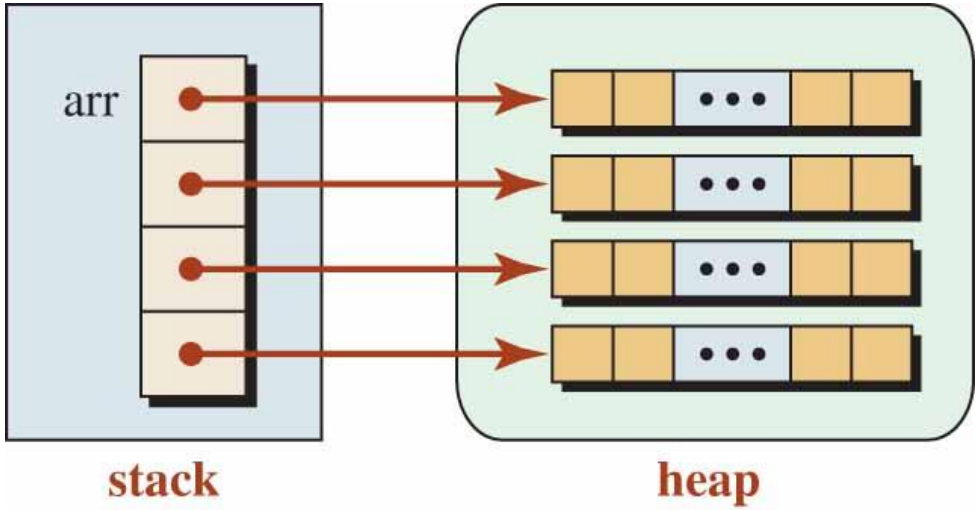
©2020 McGraw-Hill Education

Two-Dimensional Array Part 2

Using Both Stack and Heap Memory.

If the row dimension is know before the compilation, we can create an array of pointers in the stack and then create each row in the heap.

Figure 9.27 ***An array in stack and another in the heap***



©2020 McGraw-Hill Education

Two-Dimensional Array Part 3

The following shows the code to create such an array.

```
int* arr [4];  // An array of four pointers to integers in stack
cin >> colNums;
for (int i = 0; i < 4 ; i++)
{
    arr[i] = new int [colNums]; // an array of colNums in the heap
}
```

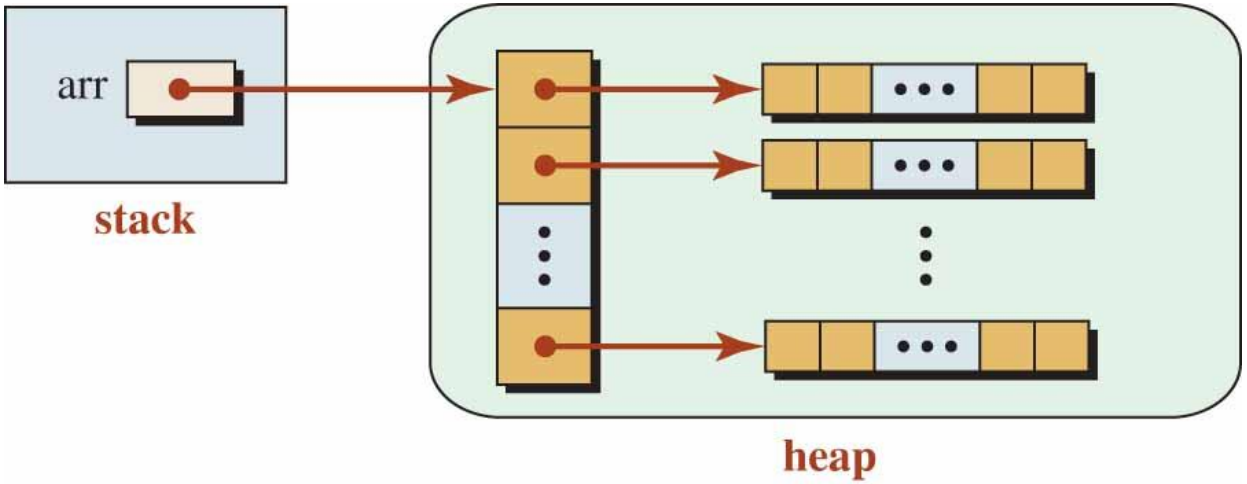
Using Only Heap Memory.

If neither of the dimensions are know before the compilation, we need to create the whole two-dimensional array in the heap as shown in Figure 9.28.

©2020 McGraw-Hill Education

Two-Dimensional Array Part 4

Figure 9.28 *A two-dimensional array totally in the heap*



The following shows the code to create such an array.

```
cin >> rowNums;
cin >> colNums;
int** arr = new int* [rowNums];
for (int i = 0; i < n + 1 ; i++)
{
    arr[i] = new int [colNums];
}
```

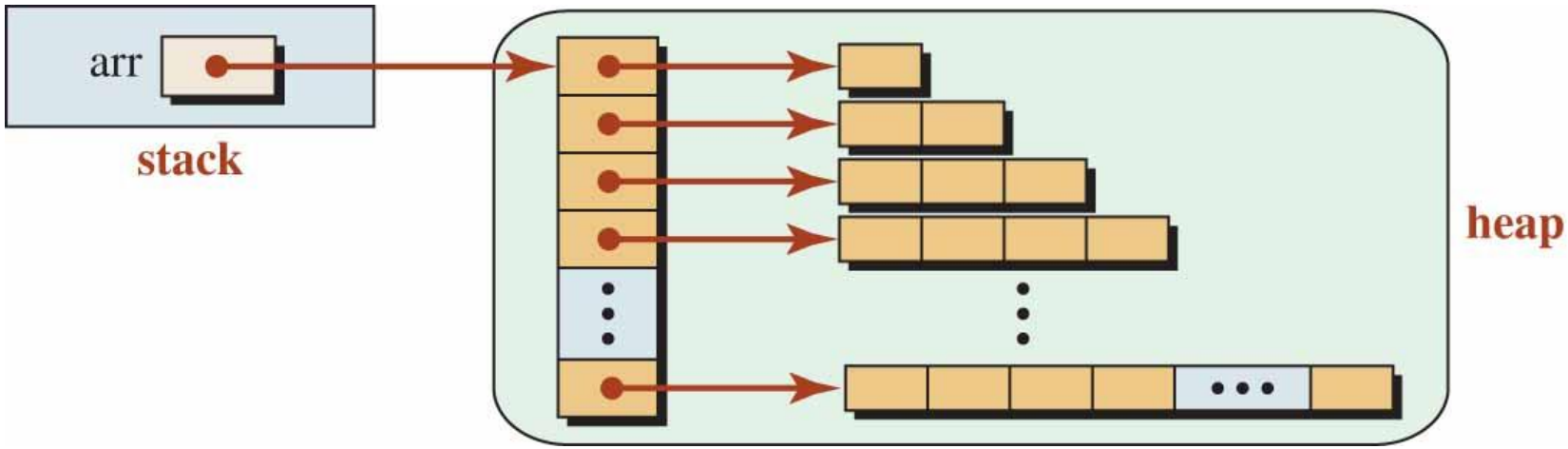

©2020 McGraw-Hill Education

Two-Dimensional Array Part 5

An Example: Ragged Array.

We can create a two-dimensional array in which the number of elements in one row is different from the number of elements in another.

Figure 9.29 ***Ragged array***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

Two-Dimensional Array Part 6

Example 9.26

Pascal's triangle determines the coefficients of a binomial expansion. When a binomial like (x + y) is raised to a positive integer power we have:

$$(x + y)^n = a_0 x^n + a_1 x^{n-1}y + a_2 x^{n-2}y^2 + \quad \ldots \quad + a_{n-1} x y^{n-1} + a_n y^n$$

The coefficient of each term (a_0, a_1, a_2, ... a_{n-1}, a_n) can be calculated using a triangle of n + 1 rows and n + 1 columns.

n = 0	1				
n = 1	1	1			
n = 2	1	2	1		
n = 3	1	3	3	1	
n = 4	1	4	6	4	1

©2020 McGraw-Hill Education

Two-Dimensional Array Part 7

Program 9.15 *Finding the Pascal coefficients*

```
/*************************************************************
 * The program shows how to create Pascal coefficients using   *
 * a ragged array dynamically allocated on the heap.           *
 *************************************************************/
#include <iostream>
#include <iomanip>
using namespace std;

int main ( )
{
    // Declaration
    int maxPower = 10;
    int n;
    // Input validation
    do
    {
        cout << "Enter the power of binomial : ";
        cin >> n;
    } while(n < 0 || n > maxPower);
    // Allocate memory from heap
```


©2020 McGraw-Hill Education

Two-Dimensional Array Part 8

Program 9.15 *Finding the Pascal coefficients*

```
21      int** pascal = new int* [n + 1];
22      for (int i = 0; i < n + 1 ; i++)
23      {
24          pascal[i] = new int [i];
25      }
26      // Formation of the coefficient
27      for (int i = 0; i <= n ; i++)
28      {
29          for (int j = 0; j < i + 1; j++)
30          {
31              if (j == 0 || i == j)
32              {
33                pascal [i][j] = 1;
34              }
35              else
36      {
37                 pascal [i][j] = pascal [i - 1] [j - 1] + pascal [i - 1][j];
38              }
39          }
40      }
```


©2020 McGraw-Hill Education

Two-Dimensional Array Part 9

Program 9.15 *Finding the Pascal coefficients*

```
    // Print coefficients
    cout << endl;
    cout << "Coefficients for (x + y)^" << n << " are:";
    for (int j = 0; j <= n ; j++)
    {
        cout << setw (5) << pascal [n][j] ;
    }
    cout << endl;
    // Delete allocated memory
    for (int i = 0; i < n + 1 ; i++)
    {
        delete [ ] pascal [i];
    }
    delete [ ] pascal;
    return 0;
}
```

```
Run:
Enter the power of binomial : 5
Coefficients for (x + y)^5 are: 1  5  10  10  5  1
```


©2020 McGraw-Hill Education

PROGRAMS DESIGN

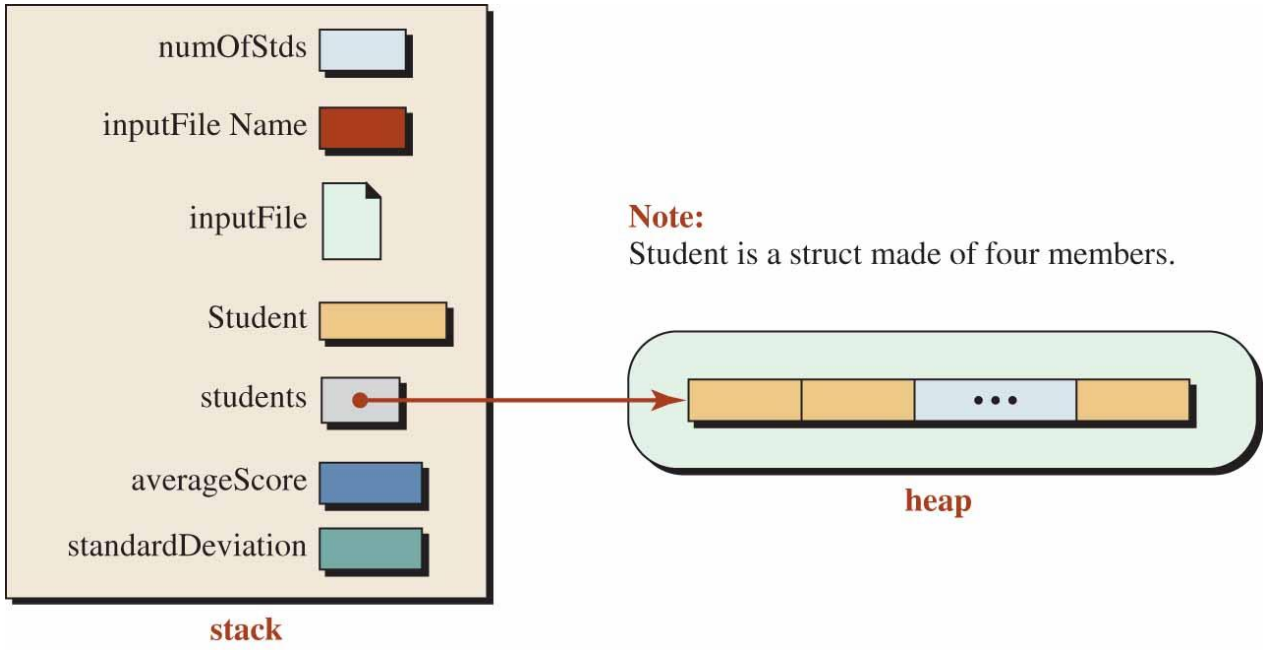
We create two classes in which some of the data members are created in the heap.

©2020 McGraw-Hill Education

A Course Class Part 1

We create a Course class whose objects can be used by a professor in a school to create statistics about each of her courses.

Figure 9.30 ***Data members of the Course class***



Access the text alternative for slide images.

©2020 McGraw-Hill Education

A Course Class Part 2

Member functions.

We have several private member functions called by the constructor automatically when an object of the class is instantiated.

The two public member functions are the constructor and destructor.

©2020 McGraw-Hill Education

A Course Class Part 3

Input File

We assume that the input file contains the following data, in which the first column represents the student identity and the second column defines the score in the course.

```
1000    88
1001    100
1002    92
1003    77
1004    54
1005    82
1006    67
1007    95
1008    93
1009    100
```


©2020 McGraw-Hill Education

A Course Class Part 4

Program 9.16 *Interface file for the Course class*

```
/*****************************************************************
 * We have private data members and public member functions.   *
 * The private member functions are helper functions called    *
 * by the constructor to do its job. The constructor is        *
 * responsible for everything. The destructor deletes arrays   *
 * created in the heap and closes the input file.              *
 *****************************************************************/
#ifndef COURSE_H
#define COURSE_H
#include <iostream>
#include <fstream>
using namespace std;

class Course
{
    private:
        int numOfStds;
        const char* inputFileName;
        ifstream inputFile;
        struct Student {int id; int score; char grade;
```


©2020 McGraw-Hill Education

A Course Class Part 5

Program 9.16 *Interface file for the Course class*

```
                                                double deviation;};
        Student* students;
        double averageScore;
        double standardDeviation;
        // Private member functions
        void getInput ();
        void setGrades ();
        void setAverage ();
        void setDeviations();
        void printResult() const;
    public:
        Course (int numOfStds, const char* inputFileName);
        ~Course ();
};
#endif
```


©2020 McGraw-Hill Education

A Course Class Part 6

Program 9.17 *Implementation file*

```
/*****************************************************************
 * The implementation file gives the definition of all private *
 * and public member functions.                                *
 *****************************************************************/
#include "course.h"
#include <iomanip>
#include <cmath>

/*****************************************************************
 * The constructor is responsible for initializing student's   *
 * number and the name of the input file containing scores.    *
 * The constructor then opens the input file and creates       *
 * an array in the heap. The rest of the job is done by helper *
 * functions that set the scores, the grades, the average and  *
 * deviations and prints the results.                          *
 *****************************************************************/
Course :: Course (int num, const char* ifn)
:numOfStds (num), inputFileName (ifn)
{
    inputFile.open (inputFileName);
```


©2020 McGraw-Hill Education

A Course Class Part 8

Program 9.17 *Implementation file*

```
21      students = new Student [numOfStds];
22      getInput ();
23      setGrades ();
24      setAverage ();
25      setDeviations();
26      printResult();
27  }
28  /*****************************************************************
29   * The destructor is reponsible for deleting the array created *
30   * in the heap using the corresponding pointer. It also closes *
31   * the input file opened by the constructor.                   *
32   *****************************************************************/
33  Course :: ~Course ()
34  {
35      delete [ ] students;
36      inputFile.close ();
37  }
38  /*****************************************************************
39   * The getInput function is responsible for reading the input  *
40   * file containg the identity and score of students.           *
```


©2020 McGraw-Hill Education

A Course Class Part 9

Program 9.17 *Implementation file*

```
 ***************************************************************/
void Course :: getInput()
{
    for (int i = 0; i < numOfStds; i++)
    {
        inputFile >> students [i].id;
        inputFile >> students [i].score;
    }
}
/***************************************************************
 * The getGrades function uses the score for each student and  *
 * changes it to the grade using an array of chars.            *
 ***************************************************************/
void Course :: setGrades()
{
    char charGrades [ ] =
          {'F', 'F', 'F' , 'F' , 'F' , 'F', 'D' , 'C' , 'B' , 'A' , 'A'};
    for (int i = 0; i < numOfStds; i++)
    {
        int index = students[i].score / 10;
```


©2020 McGraw-Hill Education

A Course Class Part 10

Program 9.17 *Implementation file*

```
61  students[i].grade = charGrades [index];
62      }
63  }
64  /*****************************************************************
65   * The setAverage function processes scores in the array and    *
66   * creates the average for the class.                            *
67   *****************************************************************/
68  void Course :: setAverage()
69  {
70      int sum = 0;
71      for (int i = 0; i < numOfStds; i++)
72      {
73          sum += students[i].score;
74      }
75      averageScore = static_cast <double> (sum) / numOfStds;
76  }
77  /*****************************************************************
78   * The setDeviation function reprocess the score to determine   *
79   * the deviation of each student's score from the average.      *
80   *****************************************************************/
```


©2020 McGraw-Hill Education

A Course Class Part 11

Program 9.17 *Implementation file*

```
void Course :: setDeviations()
{
    standardDeviation = 0.0;
    for (int i = 0; i < numOfStds; i++)
    {
        students[i].deviation = students[i].score - averageScore;
        standardDeviation += pow(students[i].deviation , 2);
    }
    standardDeviation = sqrt (standardDeviation) / numOfStds;
}
/*****************************************************************
 * The printResult function prints all information about the   *
 * the course.                                                  *
 ****************************************************************/
void Course :: printResult() const
{
    cout << endl;
    cout << "Identity Score Grade Deviation" << endl;
    cout << "-------- ----- ----- ---------" << endl;
    for (int i = 0; i < numOfStds ; i++)
```


©2020 McGraw-Hill Education

A Course Class Part 12

Program 9.17 *Implementation file*

```
    {
        cout << setw (4) << noshowpoint << noshowpos;
        cout << right << students[i].id;
        cout << setw (14) << noshowpoint << noshowpos;
        cout << right << students[i].score;
        cout << setw (10) << right << students[i].grade;
        cout << fixed << setw (20) << right << setprecision (2);
        cout << showpoint << showpos;
        cout << students[i].deviation << endl;
    }
    cout << "Average score: " << fixed << setw (4);
    cout << setprecision (2) <<averageScore << endl;
    cout << "Standard Deviation: " << standardDeviation;
}
```


©2020 McGraw-Hill Education

A Course Class Part 13

Program 9.18 *Application file*

```
/*****************************************************************
 * The application program is very simple. The user instantiates *
 * an object of the class and passes the number of students and  *
 * name of the input file where the scores are stored.           *
 * Everything is done by the constructor of the class.           *
 *****************************************************************/
#include "course.h"

int main ( )
{
    // Instantiation of the Course object
    Course course (10, "scores.dat");
    return 0;
}
```


©2020 McGraw-Hill Education

A Course Class Part 14

Program 9.18 *Application file*

```
Run:
Identity      Score     Grade     Deviation
--------      -----     -----     ---------
1000           88         B         +3.20
1001           100        A         +15.20
1002           92         A         +7.20
1003           77         C         -7.80
1004           54         F         -30.80
1005           82         B         -2.80
1006           67         D         -17.80
1007           95         A         +10.20
1008           93         A         +8.20
1009           100        A         +15.20
Average score: +84.80
Standard Deviation: +4.51
```


©2020 McGraw-Hill Education

A Matrix Class Part 1

A matrix is a table of values in mathematics. In computer programming, we can simulate a matrix with a two-dimensional array.

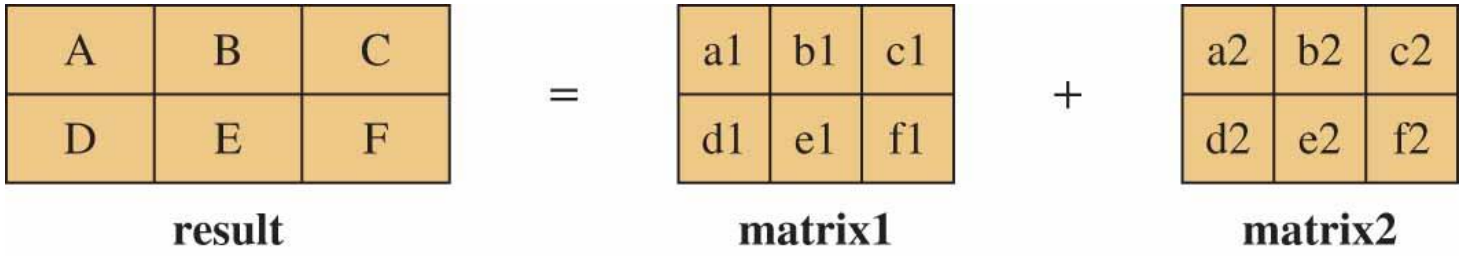
Operation

We define three operations on matrices: addition, subtraction, and multiplication. Division of matrixes is very complex and requires the inversion of matrices.

Addition

We can add two matrices if the number of rows are the same and the number of columns are the same.

Figure 9.31 ***Adding two matrices***



©2020 McGraw-Hill Education

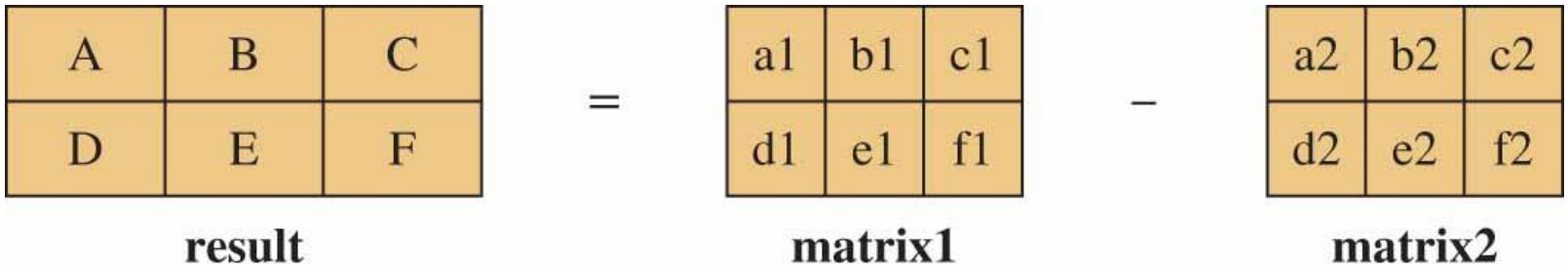
A Matrix Class Part 2

```
A = a1 + a2
...
F = f1 + f2
```

Subtraction

Subtraction is the same as addition but the values of the corresponding cells are subtracted.

Figure 9.32 ***Subtracting two matrices***




```
A = a1 - a2
...
F = f1 - f2
```


©2020 McGraw-Hill Education

A Matrix Class Part 3

Multiplication

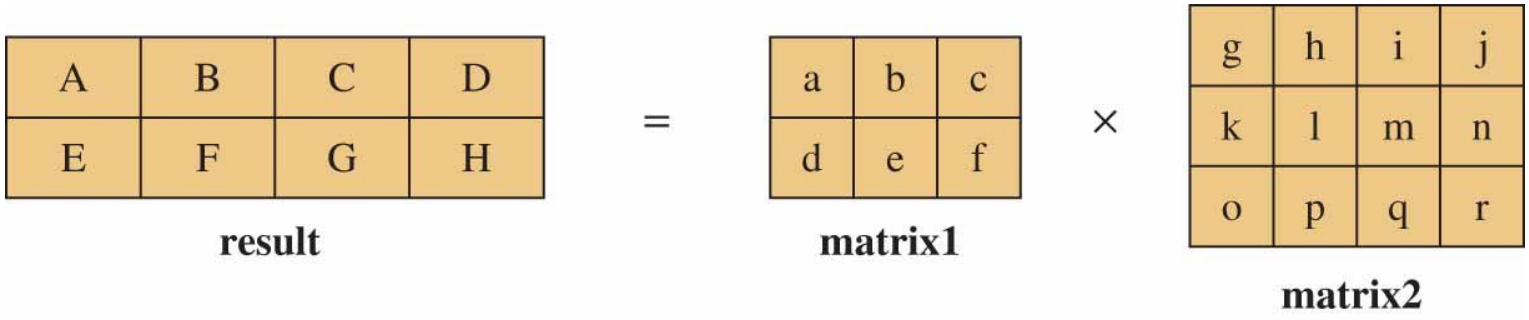
We can multiply two matrices if the number columns in the first matrix is the same as the number of rows in the second matrix (c1 == r2).

The result is a matrix with the number of rows equal r1 and the number of columns equal c2.

©2020 McGraw-Hill Education

A Matrix Class Part 4

Figure 9.33 *Multiplying two matrices*




```
A = a * g + b * k + c * o
B = a * h + b * l + c * p
...
H = d * j + e * n + f * r
```

Code

Now we define the interface, implementation, and application files.

©2020 McGraw-Hill Education

A Matrix Class Part 5

Program 9.19 *The interface file for the Matrix class*

```
/*****************************************************************
 * The interface file for the Matrix class. The only private data  *
 * members are the sizes of the matrix and the pointer that       *
 * points to the matrix in the heap.                               *
 * The constructor creates a matrix in the heap and the destructor *
 * deletes the allocated memory in the heap.                       *
 * The setup member function fills the matrices randomly           *
 * We have addition, subtraction, multiplication, and print       *
 * member functions.                                               *
 *****************************************************************/
#include <iostream>
#ifndef MATRIX_H
#define MATRIX_H
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <cassert>
using namespace std;

// Matrix class definition
```


©2020 McGraw-Hill Education

A Matrix Class Part 6

Program 9.19 *The interface file for the Matrix class*

```
21 class Matrix
22 {
23     private:
24         int rowSize;
25         int colSize;
26         int** ptr;
27     public:
28         Matrix (int rowSize, int colSize);
29         ~Matrix ();
30         void setup ();
31         void add (const Matrix& second, Matrix& result) const;
32         void subtract (const Matrix& second, Matrix& result) const;
33         void multiply (const Matrix& second, Matrix& result) const;
34         void print () const;
35 };
36 #endif
```


©2020 McGraw-Hill Education

A Matrix Class Part 7

Program 9.20 *The implementation file for the Matrix class*

```
/*****************************************************************
 * We have implemented all member functions declared in the    *
 * interface file.                                              *
 * The implementation follows the description of operations    *
 * discussed before.                                            *
 ****************************************************************/
#include "matrix.h"

// Constructor: creates a matrix in the heap
Matrix :: Matrix (int r, int c)
: rowSize (r), colSize (c)
{
    ptr = new int* [rowSize];
    for (int i = 0; i < rowSize; i++)
    {
        ptr [i] = new int [colSize];
    }
}
// Destructor: deletes memory locations in the heap
Matrix :: ~Matrix ()
```


©2020 McGraw-Hill Education

A Matrix Class Part 8

Program 9.20 *The implementation file for the Matrix class*

```
{
    for (int i = 0; i < rowSize ; i++)
    {
        delete [] ptr [i];
    }
    delete [] ptr;
}
// The setup fills the cells with random values between 1 and 5
void Matrix :: setup ()
{
    for (int i = 0; i < rowSize; i++)
    {
        for (int j = 0; j < colSize ; j++)
        {
            ptr [i][j] = rand () % 5 + 1;
        }
    }
}
// The add function adds second to the host and creates result
void Matrix :: add (const Matrix& second, Matrix& result) const
```


©2020 McGraw-Hill Education

A Matrix Class Part 9

Program 9.20 *The implementation file for the Matrix class*

```
41  {
42      assert (second.rowSize == rowSize && second.colSize == colSize);
43      assert (result.rowSize == rowSize && result.colSize == colSize);
44
45      for (int i = 0; i < rowSize ; i++)
46      {
47          for (int j = 0; j < second.colSize; j++)
48          {
49              result.ptr[i][j] = ptr[i][j] + second.ptr[i][j];
50          }
51      }
52  }
53  // The subtract function subtracts second from host
54  void Matrix :: subtract (const Matrix& second, Matrix& result) const
55  {
56      assert (second.rowSize == rowSize && second.colSize == colSize);
57      assert (result.rowSize == rowSize && result.colSize == colSize);
58      for (int i = 0; i < rowSize ; i++)
59      {
60          for (int j = 0; j < second.colSize; j++)
```


©2020 McGraw-Hill Education

Future Value of a Fixed Investment Part 6

Program 9.20 *The implementation file for the Matrix class*

```
        {
            result.ptr[i][j] = ptr[i][j] - second.ptr[i][j];
        }
    }
}
// The multiply function multiplies second by host
void Matrix :: multiply (const Matrix& second, Matrix& result) const
{
    assert (colSize == second.rowSize);
    assert (result.rowSize = rowSize);
    assert (result.colSize = second.colSize);
    for (int i = 0; i < rowSize ; i++)
    {
        for (int j = 0; j < second.colSize; j++)
        {
            result.ptr[i][j] = 0;
            for (int k = 0 ; k < colSize; k++)
            {
                result.ptr[i][j] += ptr[i][k] * second.ptr[k][j];
            }
```


©2020 McGraw-Hill Education

A Matrix Class Part 11

Program 9.20 *The implementation file for the Matrix class*

```
        }
    }
}
// The print function prints the values of cells
void Matrix :: print () const
{
    for (int i = 0 ; i < rowSize; i++)
    {
        for (int j = 0; j < colSize ; j++)
        {
            cout << setw (5) << ptr [i][j];
        }
        cout << endl;
    }
    cout << endl;
}
```


©2020 McGraw-Hill Education

A Matrix Class Part 12

Program 9.21 *Application to test the Matrix class*

```
/*****************************************************************
 * We create several matrix objects in the heap and we apply   *
 * some operations on them.                                     *
 *****************************************************************/
#include "matrix.h"

int main ()
{
     // Instantiation and setup of matrix1
     cout << "matrix1" << endl;
     Matrix matrix1 (3, 4);
     matrix1.setup ();
     matrix1.print();
     // Instantiation and setup of matrix2
     cout << "matrix2" << endl;
     Matrix matrix2 (3, 4);
     matrix2.setup ();
     matrix2.print ();
     // Instantiation and setup of matrix3
     cout << "A new matrix3" << endl;
```


©2020 McGraw-Hill Education

A Matrix Class Part 13

Program 9.21 *Application to test the Matrix class*

```
    Matrix matrix3 (4, 2);
    matrix3.setup ();
    matrix3.print ();
    // Adding matrix2 to matrix1 and printing the resulting matrix
    cout << "Result of matrix1 + matrix2" << endl;
    Matrix addResult (3, 4);
    matrix1.add (matrix2, addResult);
    addResult.print ();
    // Subtracting matrix2 from matrix1 and printing the resulting matrix
    cout << " Result of matrix1 - matrix2" << endl;
    Matrix subResult (3, 4);
    matrix1.subtract (matrix2, subResult);
    subResult.print ();
    // Multiplying matrix1 and matrix3 and printing the resulting matrix
    cout << "Result of matrix1 * matrix3"<< endl;
    Matrix mulResult (3, 2);
    matrix1.multiply (matrix3, mulResult);
    mulResult.print();
    return 0;
}
```


©2020 McGraw-Hill Education

A Matrix Class Part 14

Program 9.21 *Application to test the Matrix class*

```
Run:
matrix1
3   2   3   1
4   4   5   1
4   1   1   5
matrix2
4   3   1   3
3   5   1   5
4   5   1   4
matrix3
4   1
5   5
3   4
2   1
```


©2020 McGraw-Hill Education

A Matrix Class Part 15

Program 9.21 *Application to test the Matrix class*

```
Result of matrix1 + matrix2
             7    5    4     4
             7    9    6     6
             8    6    2     9
Result of matrix1 - matrix2
            -1   -1    2    -2
             1   -1    4    -4
             0   -4    0     1
Result of matrix1 * matrix3
            33    26
            53    45
            34    18
```


©2020 McGraw-Hill Education